SAPIENZA
UNIVERSITÀ DI ROMA

Sviluppo e simulazione di un algoritmo di Collision Avoidance tridimensionale per l'AUV Tethys

Facoltà di Informatica
Dipartimento di Ingegneria dell'informazione
Corso di laurea in Informatica

Federico Paoliani
Matricola 1911644



Relatore
Prof. Enrico Tronci



A.A. 2025/2026

Indice

Acronimi **3**

1 Introduzione **4**
1.1 Contesto 5
1.2 Motivazioni 7
1.3 Contributi 8
1.4 Lavori correlati 9
1.5 Struttura della tesi 10

2 Background **11**
2.1 Veicoli autonomi subacquei 11
2.2 Navigazione autonoma e collision avoidance 12
2.3 Ambiente ROS 2 13
2.4 Simulazione robotica con Gazebo 14
2.5 Sensori sonar e nuvole di punti 15
2.6 Algoritmo Vector Field Histogram 17
2.7 Estensione tridimensionale del Vector Field Histogram 18

3 Methods **21**
3.1 Architettura generale del sistema 21
3.2 Rappresentazione tridimensionale dell'ambiente 23
3.3 Acquisizione dei dati dai sensori sonar 24
3.4 Costruzione dell'istogramma sferico 25
3.5 Fusione dei dati provenienti dai sensori 26
3.6 Ricerca della direzione sicura 28
3.7 Controllo del veicolo subacqueo 29
3.8 Gestione degli stati di navigazione 31

4 Implementation **34**
4.1 Struttura del workspace ROS 2 34

4.2 Pacchetto `collision_avoidance` . . . 36
4.3 Nodo principale `collision_avoidance_vfh` . . . 38
4.4 Comunicazione tra ROS 2 e Gazebo . . . 40
4.5 Topic ROS 2 utilizzati . . . 41
4.6 Modello del drone subacqueo Tethys . . . 42
4.7 Configurazione dei sonar simulati . . . 43
4.8 Configurazione dei mondi simulati . . . 44
4.9 Script di avvio della simulazione . . . 45
4.10 Raccolta dei dati sperimentali . . . 47

5 Risultati sperimentali **50**
5.1 Obiettivi degli esperimenti . . . 50
5.2 Setting sperimentale . . . 51
5.3 Casi di studio . . . 52
5.3.1 Scenario con un drone . . . 54
5.3.2 Scenario con cinque droni . . . 54
5.3.3 Analisi dei fallimenti negli scenari multi-drone . . . 55
5.4 Verifica della correttezza . . . 57
5.5 Valutazione computazionale . . . 57
5.6 Valutazione tecnica dell'approccio . . . 59
5.7 Discussione dei risultati . . . 60

6 Conclusioni **62**
6.1 Sintesi del lavoro svolto . . . 63
6.2 Limiti del sistema sviluppato . . . 64
6.3 Sviluppi futuri . . . 65
6.4 Considerazioni finali . . . 66

Bibliografia **68**

Ringraziamenti **69**

Acronimi

AUV *Autonomous Underwater Vehicle*, veicolo autonomo subacqueo.

CSV *Comma-Separated Values*, formato testuale utilizzato per salvare i risultati sperimentali.

GPS *Global Positioning System*, sistema di posizionamento globale.

JSON *JavaScript Object Notation*, formato testuale usato per trasmettere le metriche durante la simulazione.

LRAUV *Long Range Autonomous Underwater Vehicle*, veicolo subacqueo autonomo a lunga autonomia.

ROS 2 *Robot Operating System 2*, framework software utilizzato per organizzare i nodi e la comunicazione del sistema robotico.

SDF *Simulation Description Format*, formato utilizzato per descrivere mondi e modelli in Gazebo.

VFH *Vector Field Histogram*, algoritmo locale per la pianificazione e l'evitamento degli ostacoli.

Capitolo 1

Introduzione

La robotica autonoma rappresenta uno degli ambiti più rilevanti nello sviluppo di sistemi capaci di operare in ambienti complessi, dinamici e difficilmente accessibili all'essere umano. Tra questi ambienti, quello subacqueo presenta caratteristiche particolarmente critiche: la visibilità può essere ridotta, le comunicazioni sono limitate, il segnale GPS non è disponibile e il veicolo deve muoversi in uno spazio tridimensionale, controllando non soltanto la direzione orizzontale, ma anche la profondità.

In questo contesto si inseriscono gli AUV, *Autonomous Underwater Vehicles*, cioè veicoli autonomi subacquei progettati per svolgere missioni senza la necessità di un controllo umano continuo. Tali veicoli possono essere utilizzati per attività di esplorazione, monitoraggio ambientale, ispezione di infrastrutture sommerse e raccolta di dati scientifici. Per svolgere queste operazioni in sicurezza, un AUV deve essere in grado di percepire l'ambiente circostante, individuare ostacoli e modificare la propria traiettoria per evitare collisioni.

Questa tesi riguarda la progettazione e simulazione di un sistema di *collision avoidance* tridimensionale per il veicolo subacqueo Tethys LRAUV. Il sistema è stato sviluppato in ambiente ROS 2 e Ignition Gazebo, utilizzando un modello simulato del drone e un insieme di sensori sonar simulati. L'obiettivo principale è permettere al veicolo di navigare autonomamente da una posizione iniziale a un obiettivo tridimensionale, evitando ostacoli presenti nell'ambiente.

Il lavoro consiste principalmente nello sviluppo del pacchetto ROS 2 `collision_avoidance`, scritto in C++, che implementa un pianificatore locale basato su una versione tridimensionale del *Vector Field Histogram*. L'algoritmo utilizza i dati provenienti dai sonar simulati per costruire una rappresentazione locale dell'ambiente, valuta la presenza di ostacoli e genera comandi per gli attuatori del drone, in particolare pinne verticali, pinne orizzontali e propulsore.

Rispetto all'algoritmo VFH originale, pensato principalmente per robot mobili che si muovono su un piano bidimensionale, il sistema sviluppato in questa tesi adatta il principio dell'istogramma angolare a un contesto tridimensionale e subacqueo. In particolare, lo spazio circostante il drone viene rappresentato mediante un istogramma sferico basato su azimuth ed elevazione, integrato con una logica di lookahead a corridoi, waypoint locali e meccanismi di protezione per la sicurezza del veicolo.

1.1 Contesto

La navigazione autonoma di un veicolo subacqueo richiede l'integrazione di diverse componenti: percezione dell'ambiente, pianificazione del movimento, controllo degli attuatori e gestione delle condizioni di emergenza. In un ambiente simulato, queste componenti devono inoltre dialogare correttamente con il motore fisico e con i modelli dei sensori e del veicolo.

Nel progetto è stato utilizzato Tethys, un veicolo autonomo subacqueo appartenente alla famiglia LRAUV, *Long Range Autonomous Underwater Vehicle*. Il modello del veicolo è stato usato come piattaforma di riferimento per integrare e testare il sistema di *collision avoidance* sviluppato durante il tirocinio. Il lavoro svolto non si è concentrato sulla progettazione fisica del drone, ma sull'integrazione di un algoritmo di navigazione locale con un modello simulato funzionante.

L'ambiente software utilizzato è composto da ROS 2 Humble e Ignition Gazebo Fortress. ROS 2 fornisce l'infrastruttura per realizzare nodi, topic e comunicazione tra componenti software, mentre Gazebo permette di simulare il mondo fisico, il veicolo, gli ostacoli, i sensori e gli attuatori. La comunicazione tra ROS 2 e Gazebo avviene tramite `ros_gz_bridge`, che consente di scambiare dati tra i topic dei due ambienti.

Il veicolo riceve informazioni dall'ambiente tramite cinque sonar simulati, implementati in Gazebo come sensori `gpu_lidar`. Questi sensori forniscono nuvole di punti tridimensionali che rappresentano gli ostacoli rilevati intorno al drone. I dati vengono elaborati dal nodo di *collision avoidance* per costruire una rappresentazione locale dello spazio e decidere la direzione di movimento più sicura.

La simulazione consente di testare il comportamento del sistema in scenari ripetibili e controllati. Nel progetto sono stati considerati scenari con un singolo drone e scenari con cinque droni. Per ciascuno scenario sono stati analizzati tre livelli di difficoltà, ottenuti aumentando la complessità dell'ambiente e il numero di ostacoli

presenti. Questa scelta ha permesso di valutare sia il comportamento del sistema al crescere della densità di ostacoli, sia l'effetto della presenza simultanea di più veicoli.

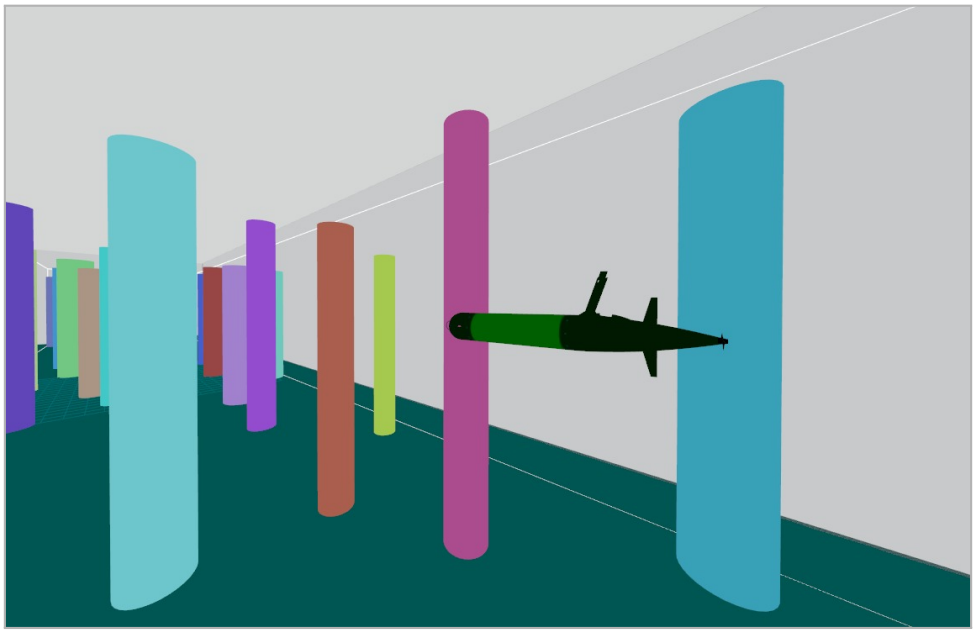


Figura 1.1. *Esempio di ambiente simulato in Ignition Gazebo con il drone Tethys durante una prova di navigazione autonoma tra ostacoli.*

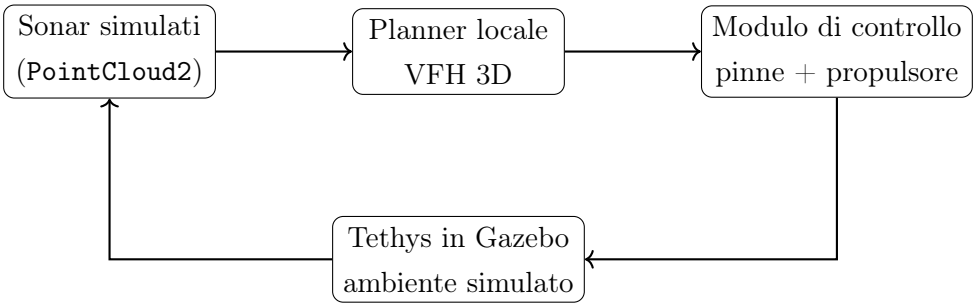


Figura 1.2. *Schema concettuale del sistema sviluppato: i sonar simulati forniscono i dati al planner locale, che genera i comandi di controllo per il drone Tethys all'interno dell'ambiente simulato.*

1.2 Motivazioni

Il problema della *collision avoidance* è centrale nella navigazione autonoma. Un veicolo che si muove senza controllo umano diretto deve essere in grado di evitare ostacoli in modo tempestivo, soprattutto quando opera in ambienti non completamente noti o difficili da modellare in anticipo.

Nel caso dei veicoli subacquei, questo problema è reso più complesso da diversi fattori. Innanzitutto, lo spazio di movimento è tridimensionale: un ostacolo può essere evitato non soltanto sterzando a destra o a sinistra, ma anche controllando la profondità del veicolo. Inoltre, la dinamica di un drone subacqueo è influenzata da effetti idrodinamici, inerzia, galleggiabilità e smorzamento del moto. Di conseguenza, una manovra decisa dall'algoritmo non produce necessariamente un effetto immediato sul movimento del veicolo.

Un ulteriore elemento di difficoltà è rappresentato dalla percezione dell'ambiente. I sensori simulati forniscono informazioni locali e limitate, e non tutti gli elementi del mondo vengono rilevati in modo affidabile. Ad esempio, alcune superfici piatte, come le pareti della vasca, possono non produrre ritorni utili nei dati dei sonar simulati. Questo richiede l'integrazione di strategie aggiuntive, come il calcolo analitico della distanza dalle pareti.

L'algoritmo *Vector Field Histogram* classico è stato pensato principalmente per robot mobili che si muovono su un piano bidimensionale. Applicare un approccio simile a un veicolo subacqueo richiede quindi un adattamento alla tridimensionalità dell'ambiente. È necessario rappresentare non solo le direzioni orizzontali, ma anche la componente verticale, e trasformare tale rappresentazione in comandi fisicamente realizzabili dal veicolo.

La motivazione del lavoro nasce quindi dalla necessità di progettare un sistema capace di combinare percezione locale, pianificazione tridimensionale e controllo del veicolo in un ambiente simulato realistico. L'obiettivo non è soltanto evitare ostacoli statici, ma anche permettere al drone di procedere verso un obiettivo, gestire situazioni di emergenza e mantenere un comportamento stabile durante la navigazione.

La valutazione sperimentale viene impostata su due scenari principali: uno con un singolo drone e uno con cinque droni. In entrambi i casi, la difficoltà viene aumentata progressivamente modificando il numero di ostacoli. Questa impostazione consente di analizzare il comportamento del sistema sia in condizioni più semplici, sia in scenari più complessi nei quali più veicoli operano contemporaneamente nello stesso ambiente.

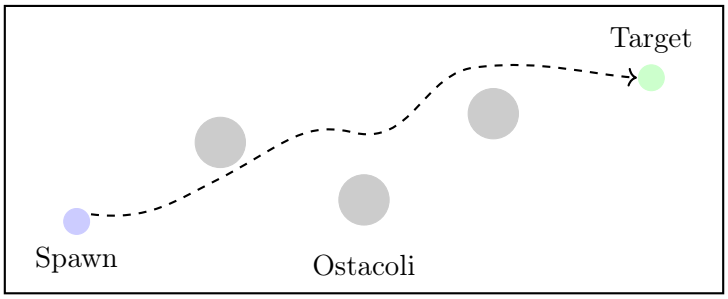


Figura 1.3. *Schema concettuale di navigazione verso il target con evitamento degli ostacoli: il drone parte da una posizione iniziale, rileva gli ostacoli presenti nell'ambiente e modifica localmente la traiettoria per raggiungere il target.*

1.3 Contributi

Il lavoro svolto durante il tirocinio ha portato alla realizzazione di un sistema di *collision avoidance* tridimensionale per il drone subacqueo Tethys simulato in ROS 2 e Gazebo.

Il primo contributo consiste nello sviluppo del pacchetto ROS 2 `collision_avoidance`, implementato in C++. Il pacchetto contiene il nodo principale di controllo, il planner locale e le strutture dati necessarie alla rappresentazione dell'ambiente. Il nodo riceve i dati dei sonar simulati, elabora la posa del veicolo, aggiorna lo stato interno del sistema e pubblica comandi verso gli attuatori del drone.

Il secondo contributo riguarda l'adattamento del principio del *Vector Field Histogram* a uno scenario tridimensionale subacqueo. Mentre il VFH classico utilizza una rappresentazione bidimensionale dell'ambiente, il sistema sviluppato impiega un istogramma sferico basato su azimuth ed elevazione. In questo modo, i punti rilevati dai sonar vengono organizzati in una griglia angolare tridimensionale che permette di valutare la pericolosità delle diverse direzioni nello spazio.

Il terzo contributo è l'integrazione della rappresentazione VFH 3D con una logica di lookahead a corridoi e waypoint locali. La scelta della manovra non dipende soltanto dal settore meno occupato dell'istogramma, ma anche dalla presenza di uno spazio percorribile davanti al drone. L'utilizzo di waypoint locali permette inoltre di rendere più stabile la manovra evasiva, evitando cambiamenti continui della direzione scelta.

Il quarto contributo riguarda la gestione dello stato di navigazione del veicolo. Il sistema distingue tra modalità di crociera, manovra evasiva e frenata di emergenza. A queste modalità si affiancano ulteriori protezioni, come il recupero della quota, il controllo dei limiti di profondità, la stabilizzazione del veicolo, la gestione della retromarcia e l'avvicinamento finale all'obiettivo.

Il quinto contributo è l'integrazione del sistema con il simulatore Gazebo e con il modello Tethys. Questo ha richiesto la configurazione dei topic ROS 2, del bridge ROS–Gazebo, dei sensori sonar, degli attuatori e degli script di avvio. Durante lo sviluppo sono stati affrontati diversi problemi pratici, tra cui la corretta ricezione dei dati sensoriali, la gestione delle pareti non rilevate dai sonar, la stabilità del veicolo durante le manovre e la raccolta delle metriche sperimentali.

Infine, il sistema è stato valutato attraverso una campagna sperimentale organizzata in due scenari principali: uno con un singolo drone e uno con cinque droni. Per ciascuno scenario sono stati considerati tre livelli di difficoltà, caratterizzati da una crescente complessità dell'ambiente. I risultati finali, discussi nel Capitolo 5, permettono di analizzare la capacità del sistema di raggiungere il target, evitare collisioni e mantenere un comportamento stabile al crescere della complessità dell'ambiente.

1.4 Lavori correlati

Il lavoro si collega a diversi ambiti della robotica autonoma: la navigazione di veicoli mobili, la *collision avoidance* locale, la simulazione robotica e la navigazione di veicoli subacquei.

Il punto di partenza del lavoro è il paper di Borenstein e Koren [1], intitolato *The Vector Field Histogram—Fast Obstacle Avoidance for Mobile Robots*. Gli autori presentano un algoritmo VFH per l'evitamento rapido degli ostacoli nei robot mobili. L'idea principale consiste nel rappresentare l'ambiente circostante tramite un istogramma e scegliere una direzione di movimento libera sulla base della distribuzione degli ostacoli. Questo approccio è adatto a sistemi che devono reagire rapidamente a ostacoli percepiti localmente, senza costruire necessariamente una mappa globale completa.

Il VFH originale è pensato per robot mobili terrestri e opera principalmente in un contesto bidimensionale. Il robot costruisce una rappresentazione dell'ambiente sul piano, identifica le direzioni occupate e seleziona una direzione di sterzata sicura. In questa tesi, il principio descritto nel paper viene adattato a un veicolo subacqueo simulato. La differenza principale è che l'ambiente non viene rappresentato solo sul piano orizzontale, ma tramite un istogramma sferico che tiene conto anche dell'elevazione.

Un secondo aspetto riguarda la simulazione robotica. Strumenti come Gazebo permettono di testare algoritmi di navigazione in ambienti virtuali prima di applicarli a sistemi reali. In questo progetto, Gazebo è stato utilizzato per simulare il mondo subacqueo, il modello Tethys, i sensori sonar e gli attuatori. L'utilizzo

della simulazione ha reso possibile eseguire numerose prove in scenari controllati e ripetibili.

Un terzo ambito riguarda ROS 2, utilizzato come framework per l'organizzazione del sistema robotico. ROS 2 consente di separare il software in nodi indipendenti che comunicano tramite topic. Questa struttura si adatta bene al progetto, poiché permette di separare il simulatore, il bridge di comunicazione, il nodo di *collision avoidance*, i sensori e gli attuatori.

Infine, il lavoro si collega alla navigazione autonoma di AUV. Molti sistemi subacquei richiedono capacità di percezione locale e reazione agli ostacoli, specialmente in ambienti nei quali non è disponibile una mappa completa. Il progetto si differenzia dagli approcci puramente teorici perché integra l'algoritmo in un sistema simulato funzionante, con sensori, attuatori, vincoli fisici, gestione degli stati e raccolta di metriche sperimentali.

1.5 Struttura della tesi

La tesi è organizzata come segue.

Il Capitolo 2 introduce il background necessario alla comprensione del lavoro. Vengono descritti i veicoli autonomi subacquei, il problema della navigazione autonoma, l'ambiente ROS 2, la simulazione con Gazebo, i sensori sonar, le nuvole di punti e l'algoritmo *Vector Field Histogram*.

Il Capitolo 3 descrive il metodo proposto. Viene presentata l'architettura generale del sistema, la rappresentazione tridimensionale dell'ambiente, la costruzione dell'istogramma sferico, la fusione dei dati provenienti dai sensori, la ricerca della direzione sicura, il controllo del veicolo e la gestione degli stati di navigazione.

Il Capitolo 4 descrive l'implementazione del sistema. Vengono illustrati la struttura del workspace ROS 2, il pacchetto `collision_avoidance`, il nodo principale, i topic utilizzati, la comunicazione tra ROS 2 e Gazebo, la configurazione del modello Tethys, gli script di avvio e la raccolta dei dati sperimentali.

Il Capitolo 5 presenta i risultati sperimentali. Vengono descritti gli obiettivi degli esperimenti, il setting utilizzato, i casi di studio, le verifiche di correttezza, la valutazione computazionale e la valutazione tecnica dell'approccio. In particolare, la valutazione è organizzata su due scenari principali, con un singolo drone e con cinque droni, ciascuno analizzato su tre livelli di difficoltà.

Il Capitolo 6 conclude la tesi riassumendo il lavoro svolto, discutendo i limiti del sistema sviluppato e indicando possibili sviluppi futuri.

Capitolo 2

Background

In questo capitolo vengono introdotti i concetti teorici e tecnologici necessari per comprendere il lavoro svolto durante il tirocinio. Il progetto riguarda la progettazione e la simulazione di un sistema di *collision avoidance* tridimensionale per il veicolo subacqueo Tethys LRAUV, sviluppato in ambiente ROS 2 e Ignition Gazebo.

Prima di descrivere nel dettaglio il metodo proposto e la sua implementazione, è utile fornire una panoramica sui veicoli autonomi subacquei, sulla navigazione autonoma, sugli strumenti software utilizzati, sui sensori sonar e sull'algoritmo *Vector Field Histogram*, che costituisce il principale riferimento teorico da cui prende spunto il lavoro.

2.1 Veicoli autonomi subacquei

I veicoli autonomi subacquei, indicati comunemente con l'acronimo AUV, *Autonomous Underwater Vehicles*, sono sistemi robotici progettati per operare in ambiente subacqueo senza la necessità di un controllo umano continuo. A differenza dei veicoli comandati da remoto, gli AUV devono essere in grado di prendere decisioni durante la missione, adattandosi alle condizioni dell'ambiente circostante.

Questi veicoli possono essere impiegati in diversi contesti applicativi, come l'esplorazione dei fondali marini, il monitoraggio ambientale, l'ispezione di infrastrutture sommerse, la ricerca scientifica e la raccolta di dati oceanografici. In tali scenari, il veicolo deve spesso muoversi in ambienti complessi, nei quali possono essere presenti ostacoli statici, visibilità ridotta e difficoltà di comunicazione con l'esterno.

La navigazione subacquea presenta problematiche specifiche rispetto alla navigazione terrestre o aerea. In primo luogo, il segnale GPS non è normalmente disponibile sott'acqua, rendendo più complessa la localizzazione del veicolo. Inoltre, l'ambiente subacqueo è tridimensionale: il drone non deve soltanto muoversi in avanti, a destra

o a sinistra, ma deve anche controllare la propria quota, evitando sia il fondale sia eventuali ostacoli posti sopra o sotto la sua traiettoria.

Nel presente lavoro è stato utilizzato il modello simulato del veicolo subacqueo Tethys, appartenente alla famiglia LRAUV, *Long Range Autonomous Underwater Vehicle*. Il modello è stato usato come piattaforma di riferimento per integrare e testare il sistema di *collision avoidance* sviluppato durante il tirocinio.

Il controllo del veicolo avviene attraverso i suoi principali attuatori. Le pinne verticali permettono di modificare la direzione orizzontale del drone, agendo sullo yaw. Le pinne orizzontali influenzano il pitch e quindi il controllo della profondità. Il propulsore genera la spinta necessaria al movimento in avanti o, in alcune condizioni, alla retromarcia controllata. L'integrazione tra questi elementi consente al sistema di trasformare le decisioni dell'algoritmo in comandi effettivi di movimento.

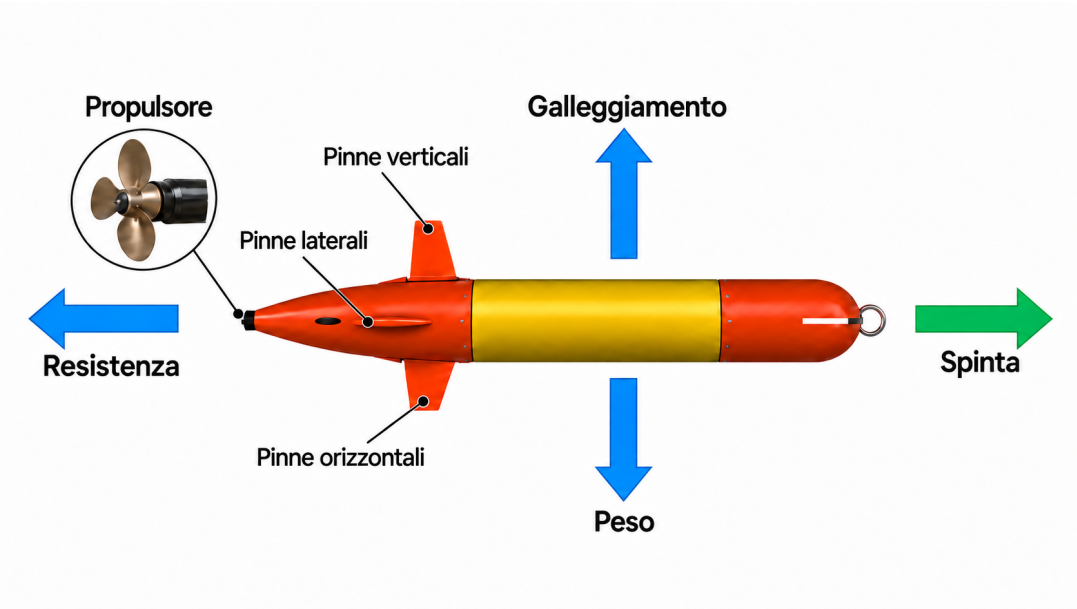


Figura 2.1. *Schema delle principali forze e componenti del veicolo subacqueo Tethys: resistenza, spinta, galleggiamento, peso, propulsore e pinne di controllo.*

2.2 Navigazione autonoma e collision avoidance

La navigazione autonoma è la capacità di un robot di muoversi all'interno di un ambiente senza intervento diretto da parte di un operatore umano. Essa comprende diversi problemi, tra cui la percezione dell'ambiente, la localizzazione, la pianificazione del percorso e il controllo del movimento. Una componente fondamentale della navigazione autonoma è la *collision avoidance*, cioè la capacità del robot di evitare ostacoli durante il moto.

Il problema della *collision avoidance* può essere affrontato a diversi livelli. In alcuni casi, il robot dispone di una mappa globale dell'ambiente e può pianificare un percorso completo dal punto di partenza al punto di arrivo. In altri casi, invece, il robot deve reagire localmente agli ostacoli rilevati dai sensori, anche senza conoscere l'intera struttura dell'ambiente. Quest'ultimo approccio è particolarmente utile in scenari dinamici, poco conosciuti o difficili da mappare in modo completo.

Nel caso di un drone subacqueo, la *collision avoidance* deve tenere conto della natura tridimensionale dell'ambiente. Un ostacolo può trovarsi davanti al veicolo, lateralmente, sopra o sotto di esso. Di conseguenza, un algoritmo pensato esclusivamente per un piano bidimensionale non è sufficiente a descrivere tutte le possibili manovre disponibili. Il veicolo deve infatti poter decidere se aggirare un ostacolo lateralmente, cambiare quota, rallentare oppure eseguire una manovra di emergenza.

Nel lavoro svolto, il sistema di navigazione è stato progettato come un planner locale. Il veicolo non costruisce una mappa globale completa dell'ambiente, ma utilizza le informazioni provenienti dai sensori per scegliere, a ogni ciclo di controllo, una direzione di movimento considerata sicura. L'obiettivo è permettere al drone di avanzare verso un target tridimensionale riducendo il rischio di collisione.

Questo approccio richiede la combinazione di tre elementi principali: una rappresentazione locale dell'ambiente, una logica per selezionare una direzione libera e un sistema di controllo capace di trasformare tale decisione in comandi per il veicolo. Nel progetto questi elementi sono stati realizzati tramite un istogramma sferico tridimensionale, un planner locale ispirato al VFH e una macchina a stati che gestisce navigazione ordinaria, manovre evasive e frenata.

2.3 Ambiente ROS 2

ROS 2, *Robot Operating System 2*, è un framework software utilizzato per lo sviluppo di applicazioni robotiche. Non si tratta di un sistema operativo in senso stretto, ma di un insieme di librerie, strumenti e convenzioni che permettono di organizzare un sistema robotico complesso in più componenti indipendenti, in grado di comunicare tra loro.

Nel contesto di questo progetto, ROS 2 è stato utilizzato per implementare la logica di controllo del drone subacqueo e per gestire lo scambio di informazioni tra l'algoritmo di *collision avoidance* e l'ambiente di simulazione. Il sistema è stato sviluppato utilizzando ROS 2 Humble su Ubuntu, in integrazione con Ignition Gazebo Fortress.

Uno dei concetti fondamentali di ROS 2 è quello di nodo. Un nodo è un processo software responsabile di una specifica funzionalità del sistema. In un'applicazione robotica possono esistere diversi nodi: alcuni dedicati alla lettura dei sensori, altri al controllo degli attuatori, altri ancora alla pianificazione o alla visualizzazione dei dati.

La comunicazione tra nodi avviene principalmente tramite topic. Un topic è un canale di comunicazione sul quale un nodo può pubblicare messaggi e al quale altri nodi possono sottoscriversi. Il nodo che invia dati viene chiamato publisher, mentre il nodo che riceve dati viene chiamato subscriber. Questa architettura permette di separare le diverse parti del sistema, rendendo il software più modulare e più semplice da modificare.

Nel progetto sviluppato, il pacchetto principale realizzato è `collision_avoidance`. Esso contiene il nodo responsabile dell'elaborazione dei dati sensoriali e della generazione dei comandi di controllo per il veicolo. Il nodo riceve in ingresso le informazioni provenienti dai sonar simulati, rappresentate come nuvole di punti, e utilizza tali dati per costruire una rappresentazione tridimensionale locale dell'ambiente.

In uscita, il nodo pubblica comandi destinati alle pinne verticali, alle pinne orizzontali e al propulsore del drone. Inoltre, il sistema utilizza messaggi relativi alla posa del veicolo e ai contatti fisici per aggiornare lo stato interno e raccogliere metriche sperimentali.

2.4 Simulazione robotica con Gazebo

Gazebo è un simulatore robotico utilizzato per riprodurre ambienti fisici virtuali nei quali testare robot, sensori e algoritmi di controllo. Nel presente lavoro è stato utilizzato Ignition Gazebo Fortress per simulare il comportamento del veicolo subacqueo Tethys all'interno di ambienti controllati.

La simulazione rappresenta un passaggio fondamentale nello sviluppo di sistemi robotici autonomi. Prima di testare un algoritmo su un robot reale, è infatti utile verificarne il funzionamento in un ambiente virtuale, dove è possibile riprodurre scenari complessi, modificare facilmente gli ostacoli, ripetere gli esperimenti e osservare il comportamento del robot in condizioni diverse.

Nel caso del drone subacqueo, Gazebo consente di simulare non solo la presenza del veicolo, ma anche l'interazione con l'ambiente circostante. Nel progetto sono stati definiti mondi simulati contenenti una vasca, ostacoli cilindrici, scenari a corridoio e configurazioni multi-drone.

Gli ambienti sono descritti mediante file SDF, *Simulation Description Format*. Questi file permettono di specificare gli elementi presenti nella scena, come modelli, luci, pareti, ostacoli, proprietà fisiche e plugin di simulazione. Nel progetto, i file SDF definiscono sia il mondo simulato sia il modello del veicolo Tethys e dei suoi sensori.

Un aspetto importante dell'integrazione tra ROS 2 e Gazebo è la comunicazione tra i due ambienti. Gazebo gestisce la simulazione fisica, i sensori e gli attuatori simulati, mentre ROS 2 esegue il nodo di controllo sviluppato. Per permettere lo scambio di dati tra questi due sistemi è stato utilizzato `ros_gz_bridge`.

Attraverso il bridge, i dati generati dai sonar simulati vengono resi disponibili al nodo ROS 2 sotto forma di messaggi `PointCloud2`. Allo stesso modo, i comandi prodotti dall'algoritmo di *collision avoidance* vengono inviati agli attuatori simulati del drone. In questo modo, il sistema sviluppato può osservare l'ambiente, scegliere una direzione di movimento e inviare comandi al veicolo all'interno della simulazione.

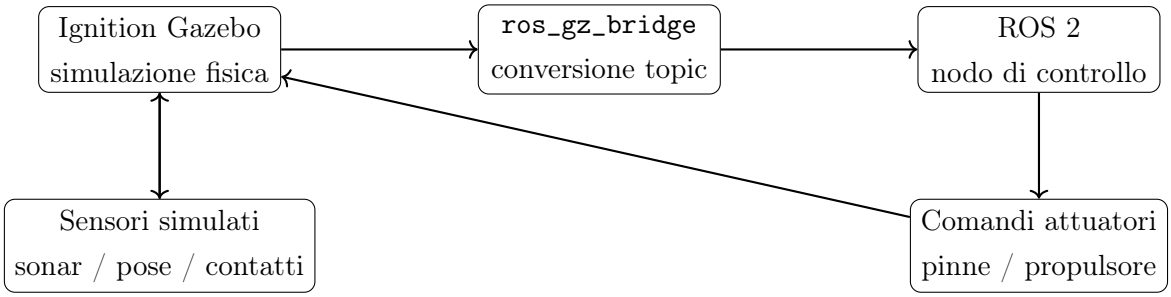


Figura 2.2. *Schema semplificato della comunicazione tra Ignition Gazebo,* `ros_gz_bridge` *e il nodo ROS 2 di controllo.*

2.5 Sensori sonar e nuvole di punti

Per permettere a un veicolo subacqueo autonomo di evitare ostacoli, è necessario dotarlo di sensori in grado di percepire l'ambiente circostante. In ambiente subacqueo, l'utilizzo di sensori ottici può risultare limitato dalla scarsa visibilità, dalla torbidità dell'acqua e dalle condizioni di illuminazione. Per questo motivo, nei sistemi subacquei vengono spesso utilizzati sensori acustici, come i sonar.

Nel progetto sviluppato, i sonar sono stati simulati in Gazebo mediante sensori di tipo `gpu_lidar`. Sebbene il sensore sia implementato tecnicamente come lidar all'interno del simulatore, esso viene utilizzato per rappresentare un comportamento analogo a quello di un sonar, cioè per rilevare ostacoli presenti nello spazio circostante al veicolo.

I dati prodotti dai sensori vengono trasmessi al nodo ROS 2 sotto forma di messaggi `sensor_msgs/msg/PointCloud2`. Una nuvola di punti, o *point cloud*, è una rappresentazione dell'ambiente costituita da un insieme di punti nello spazio tridimensionale. Ogni punto è descritto tramite coordinate cartesiane, generalmente indicate come x, y e z, e rappresenta una possibile superficie rilevata dal sensore.

Nella versione finale del sistema sono stati utilizzati cinque sonar simulati, orientati in direzioni diverse. Questa configurazione consente al drone di osservare non soltanto la zona frontale, ma anche le aree laterali, superiore e inferiore. In questo modo il sistema dispone di una percezione più completa dell'ambiente locale.

Sensore	**Campioni**	**Frequenza**	**Portata**
Frontale	30×15	5 Hz	35 m
Sinistro	10×5	2 Hz	10 m
Destro	10×5	2 Hz	10 m
Inferiore	1 raggio	2 Hz	10 m
Superiore	5×5	2 Hz	10 m

Tabella 2.1. *Configurazione dei sonar simulati utilizzati dal veicolo Tethys.*

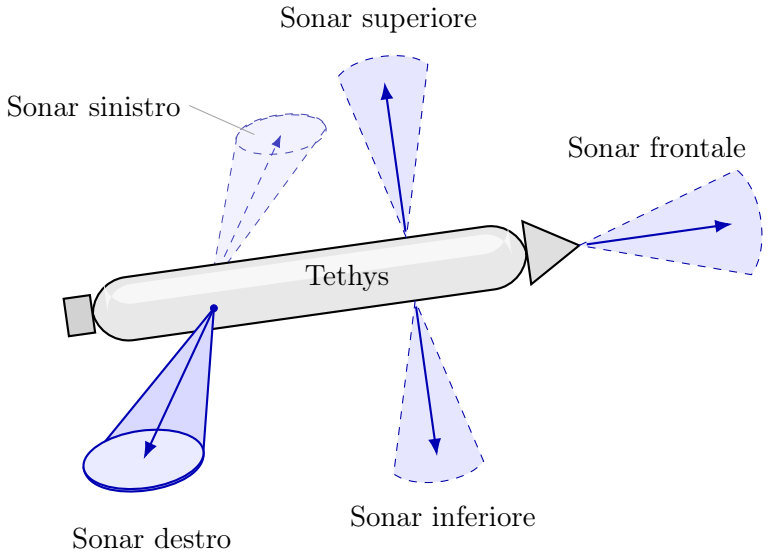


Figura 2.3. *Schema concettuale della disposizione dei sonar simulati attorno al veicolo Tethys. Il sonar frontale osserva la direzione di avanzamento, il sonar superiore e quello inferiore controllano la componente verticale, mentre il sonar destro è rappresentato come cono uscente verso l'osservatore.*

Il sonar frontale rappresenta il sensore principale per l'individuazione degli ostacoli posti lungo la direzione di avanzamento. I sonar laterali contribuiscono invece a valutare la presenza di ostacoli a sinistra e a destra, migliorando la scelta della

direzione di fuga. Il sonar superiore e quello inferiore forniscono informazioni utili per il controllo tridimensionale della traiettoria e per la gestione della quota.

Le informazioni provenienti dai sonar vengono fuse all'interno dell'algoritmo di pianificazione locale. Ogni punto rilevato viene analizzato in base alla sua posizione rispetto al drone e contribuisce alla costruzione di una rappresentazione tridimensionale dell'ambiente. Tale rappresentazione costituisce l'input principale per il modulo di *collision avoidance*.

Un limite emerso durante lo sviluppo riguarda la rilevazione delle pareti piatte della vasca. Tali superfici non vengono sempre percepite in modo affidabile dai sensori simulati. Per questo motivo, oltre ai dati provenienti dai sonar, il sistema utilizza anche un calcolo analitico della distanza dalle pareti, basato sulla posizione e sull'orientamento del veicolo.

2.6 Algoritmo Vector Field Histogram

L'algoritmo *Vector Field Histogram*, spesso abbreviato in VFH, è un metodo di *obstacle avoidance* locale utilizzato in robotica mobile. L'idea principale dell'algoritmo consiste nel rappresentare l'ambiente circostante al robot mediante un istogramma, nel quale ogni settore indica la presenza o meno di ostacoli in una certa direzione.

Il metodo originale è stato proposto da Borenstein e Koren [1] per robot mobili terrestri e si basa su una rappresentazione bidimensionale dell'ambiente. Il robot utilizza i dati dei sensori per costruire una rappresentazione locale dello spazio attorno a sé, riduce tale rappresentazione a un istogramma polare e seleziona una direzione di sterzata che consenta di evitare gli ostacoli mantenendo il movimento verso il target.

Sulla base di questa rappresentazione, il robot sceglie una direzione di movimento che risulti sufficientemente libera e compatibile con l'obiettivo di navigazione. In questo senso, il VFH è un algoritmo reattivo: il robot non ha necessariamente bisogno di una mappa globale completa, ma prende decisioni a partire dalle informazioni sensoriali locali.

Nella sua formulazione classica, il VFH è pensato per robot mobili che si muovono su un piano bidimensionale. L'ambiente viene suddiviso in settori angolari, generalmente riferiti all'angolo orizzontale rispetto alla direzione frontale del robot. A ogni settore viene associato un valore che rappresenta il livello di occupazione o pericolosità di quella direzione.

Il funzionamento generale del VFH può essere riassunto in tre fasi principali. In primo luogo, il robot acquisisce dati dai sensori e li utilizza per costruire una

rappresentazione locale dell'ambiente. In secondo luogo, tale rappresentazione viene trasformata in un istogramma polare, nel quale gli ostacoli contribuiscono ad aumentare il peso dei settori corrispondenti. Infine, l'algoritmo ricerca una direzione libera, cioè una zona dell'istogramma con basso livello di occupazione, e seleziona una direzione di movimento sicura.

Un aspetto importante del VFH è il fatto che gli ostacoli più vicini devono avere maggiore influenza rispetto a quelli lontani. Per questo motivo, nel sistema sviluppato, il contributo di un ostacolo viene pesato in funzione della sua distanza dal veicolo. In particolare, viene utilizzato un peso inversamente proporzionale al quadrato della distanza. In questo modo, un ostacolo molto vicino produce un valore elevato nell'istogramma, mentre un ostacolo lontano ha un impatto minore sulla scelta della traiettoria.

Il VFH è particolarmente adatto a sistemi che devono reagire rapidamente agli ostacoli percepiti localmente. Tuttavia, l'algoritmo classico presenta un limite importante rispetto al caso affrontato in questo lavoro: esso opera principalmente su un piano bidimensionale. Un drone subacqueo, invece, si muove in uno spazio tridimensionale e deve poter scegliere non soltanto una deviazione laterale, ma anche valutare la componente verticale e la quota del veicolo.

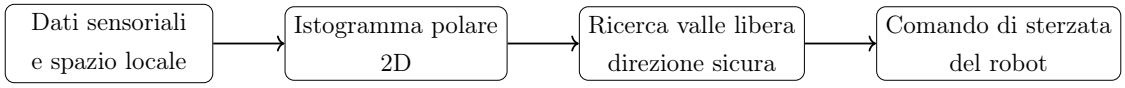


Figura 2.4. *Schema concettuale del VFH classico: i dati sensoriali vengono ridotti a un istogramma polare bidimensionale, dal quale viene selezionata una direzione di movimento.*

2.7 Estensione tridimensionale del Vector Field Histogram

Per adattare l'approccio VFH al caso subacqueo, nel presente lavoro è stata utilizzata una rappresentazione basata su un istogramma sferico tridimensionale. Invece di suddividere soltanto il piano orizzontale in settori angolari, lo spazio attorno al drone viene suddiviso secondo due angoli: azimuth ed elevazione.

L'azimuth rappresenta la direzione orizzontale rispetto al veicolo, mentre l'elevazione rappresenta la componente verticale della direzione osservata. Ogni punto rilevato dai sonar viene inizialmente espresso in coordinate cartesiane e successivamente associato a una direzione nello spazio. Tale direzione permette di individuare il settore dell'istogramma sferico in cui inserire il contributo del punto.

L'istogramma utilizzato nel sistema è composto da una griglia di 36 settori di azimuth e 18 settori di elevazione. Ogni settore rappresenta un intervallo angolare di circa 10 gradi. Questa discretizzazione consente di ottenere una rappresentazione compatta dell'ambiente locale, mantenendo allo stesso tempo un livello di dettaglio sufficiente per distinguere le principali direzioni libere e occupate.

A ogni settore dell'istogramma viene associato un valore che indica il livello di occupazione della corrispondente direzione. Quando un punto rilevato dai sonar ricade all'interno di un determinato settore, il peso di quel settore viene aumentato. Il contributo del punto dipende dalla sua distanza: gli ostacoli più vicini hanno un peso maggiore rispetto a quelli più lontani.

Oltre al valore complessivo di occupazione, per ogni settore viene mantenuta anche l'informazione sulla distanza minima dell'ostacolo rilevato in quella direzione. Questo permette al planner non soltanto di sapere quali direzioni risultano più occupate, ma anche quanto siano vicini gli ostacoli più pericolosi.

Per ridurre oscillazioni e variazioni improvvise nella scelta della direzione, l'istogramma viene sottoposto a una fase di smoothing. Lo smoothing permette di distribuire parzialmente il peso di un settore anche sui settori vicini, rendendo più continua la rappresentazione dell'ambiente.

Una volta costruito l'istogramma sferico, il planner ricerca una direzione sicura. Nel sistema sviluppato, questa ricerca non si limita alla scelta del settore meno occupato, ma tiene conto anche della presenza di un corridoio percorribile davanti al drone. Per questo motivo, l'algoritmo integra una logica di lookahead a corridoi e waypoint locali.

L'estensione tridimensionale del VFH consente al drone di mantenere una rappresentazione più ricca dell'ambiente rispetto a un approccio puramente bidimensionale. In particolare, il sistema considera la distribuzione degli ostacoli nello spazio, la distanza minima nelle varie direzioni, la posizione del target, la distanza dalle pareti e lo stato dinamico del veicolo.

Nel progetto sviluppato, la rappresentazione sferica dell'ambiente viene integrata in una macchina a stati composta principalmente da tre modalità: `CRUISE`, `ESCAPE` e `BRAKE`. La modalità `CRUISE` rappresenta la navigazione ordinaria verso il target; la modalità `ESCAPE` viene utilizzata per eseguire una manovra evasiva; la modalità `BRAKE` interviene nelle situazioni più critiche, quando è necessario ridurre rapidamente il rischio di collisione.

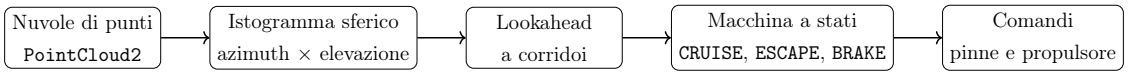


Figura 2.5. *Flusso logico dell'adattamento tridimensionale del VFH: dalle nuvole di punti all'istogramma sferico, fino alla scelta dei comandi di controllo.*

L'approccio descritto costituisce quindi un adattamento del principio del Vector Field Histogram a un contesto tridimensionale e subacqueo. La rappresentazione sferica, la fusione dei dati provenienti da più sonar, il lookahead a corridoi e la gestione degli stati di navigazione permettono al sistema di selezionare manovre più adatte al comportamento di un AUV rispetto a un algoritmo bidimensionale tradizionale.

Capitolo 3

Methods

In questo capitolo viene descritta la metodologia adottata per progettare il sistema di *collision avoidance* tridimensionale per il veicolo subacqueo Tethys. L'obiettivo non è ancora quello di descrivere i dettagli implementativi del codice, che verranno trattati nel capitolo successivo, ma di presentare l'architettura generale del sistema, il flusso dei dati, la rappresentazione dell'ambiente e la logica utilizzata per scegliere una manovra sicura.

Il sistema sviluppato è basato su un approccio di pianificazione locale. Il drone non utilizza una mappa globale completa dell'ambiente, ma prende decisioni a partire dalle informazioni percepite localmente tramite i sensori sonar simulati. I dati provenienti dai sensori vengono trasformati in una rappresentazione tridimensionale compatta, sotto forma di istogramma sferico, e successivamente utilizzati per individuare una direzione di movimento che riduca il rischio di collisione.

Il metodo combina percezione locale, fusione dei dati sensoriali, rappresentazione tridimensionale dell'ambiente, ricerca di direzioni sicure e controllo del veicolo. Il comportamento complessivo del sistema è organizzato tramite una macchina a stati, che consente di distinguere tra navigazione ordinaria, manovre evasive e condizioni di emergenza.

3.1 Architettura generale del sistema

L'architettura generale del sistema può essere suddivisa in quattro componenti principali: ambiente di simulazione, sensori, planner locale e modulo di controllo del veicolo.

L'ambiente di simulazione è realizzato in Ignition Gazebo e contiene il modello del drone subacqueo Tethys, la vasca, gli ostacoli e gli eventuali altri droni presenti nello

scenario. Gazebo si occupa della simulazione fisica del veicolo, della generazione dei dati sensoriali e della risposta del modello ai comandi inviati agli attuatori.

I sensori simulati sono costituiti da cinque sonar modellati tramite sensori `gpu_lidar`. Essi forniscono al sistema informazioni sull'ambiente circostante sotto forma di nuvole di punti tridimensionali. La presenza di più sonar permette di osservare non solo la zona frontale del drone, ma anche le aree laterali, superiore e inferiore.

Il planner locale rappresenta il nucleo decisionale del sistema. Esso riceve i dati provenienti dai sonar, li elabora e costruisce un istogramma sferico tridimensionale. Tale istogramma viene usato per stimare quali direzioni siano libere e quali, invece, siano occupate o pericolose a causa della presenza di ostacoli.

Il modulo di controllo traduce la direzione scelta dal planner in comandi per gli attuatori del veicolo. In particolare, vengono controllate le pinne verticali, le pinne orizzontali e il propulsore. Le pinne verticali permettono di modificare la direzione orizzontale del drone, le pinne orizzontali influenzano quota e pitch, mentre il propulsore regola la spinta.

Il flusso generale del sistema può essere riassunto nel modo seguente:

1. i sonar simulati acquisiscono informazioni sull'ambiente;
2. i dati vengono convertiti in nuvole di punti;
3. il nodo ROS 2 riceve i dati tramite il bridge ROS–Gazebo;
4. il planner costruisce una rappresentazione tridimensionale locale;
5. viene individuata una direzione libera o una manovra evasiva;
6. il modulo di controllo genera i comandi per pinne e propulsore;
7. Gazebo aggiorna il movimento del drone nella simulazione.

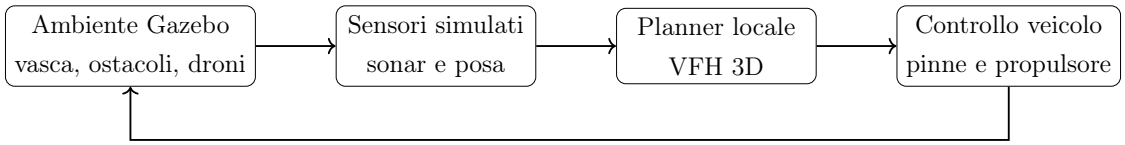


Figura 3.1. *Architettura concettuale del sistema e ciclo reattivo di percezione, pianificazione e controllo: il drone percepisce l'ambiente simulato, elabora i dati tramite il planner locale e invia comandi agli attuatori.*

Questa architettura consente di separare chiaramente la percezione, la pianificazione e il controllo. Tale separazione rende il sistema più modulare e permette di modificare singole componenti senza dover riprogettare l'intera struttura.

3.2 Rappresentazione tridimensionale dell'ambiente

Poiché il veicolo opera in ambiente subacqueo, la rappresentazione dell'ambiente deve tenere conto della tridimensionalità dello spazio. Un ostacolo può trovarsi davanti al drone, lateralmente, sopra o sotto la sua traiettoria. Per questo motivo, una rappresentazione bidimensionale non sarebbe sufficiente a descrivere in modo completo le possibili alternative di movimento.

Nel sistema sviluppato, l'ambiente locale viene rappresentato mediante un istogramma sferico tridimensionale. Lo spazio attorno al veicolo viene discretizzato in settori angolari definiti da due coordinate: azimuth ed elevazione. L'azimuth rappresenta la direzione sul piano orizzontale, mentre l'elevazione rappresenta l'inclinazione verticale della direzione considerata.

La versione finale dell'algoritmo utilizza una griglia composta da 36 settori di azimuth e 18 settori di elevazione. Ogni settore copre un intervallo angolare di circa 10 gradi. Questa scelta rappresenta un compromesso tra precisione e semplicità computazionale: una griglia troppo fine aumenterebbe il costo di elaborazione, mentre una griglia troppo grossolana ridurrebbe la capacità del sistema di distinguere direzioni libere vicine tra loro.

Ogni settore dell'istogramma contiene informazioni sulla presenza di ostacoli nella direzione corrispondente. Quando un punto rilevato dai sonar cade all'interno di un settore, il valore di occupazione di quel settore viene incrementato. In questo modo, direzioni con molti ostacoli o con ostacoli molto vicini assumono valori elevati, mentre direzioni libere mantengono valori bassi.

Oltre al valore di occupazione, viene mantenuta anche la distanza minima rilevata per ciascun settore. Questa informazione è utile per distinguere tra una direzione leggermente occupata ma lontana e una direzione in cui è presente un ostacolo molto vicino. La distanza minima diventa quindi un parametro importante per l'attivazione di manovre evasive o di emergenza.

La rappresentazione tridimensionale non implica necessariamente che il veicolo esegua sempre una fuga verticale. Nei mondi sperimentali attuali, la fuga verticale esplicita è disattivata, mentre restano attivi il controllo della quota, le protezioni sui limiti di profondità e l'elaborazione tridimensionale dei dati sensoriali. In questo modo, il sistema mantiene una percezione 3D dell'ambiente, pur privilegiando manovre evasive basate su corridoi e waypoint locali.

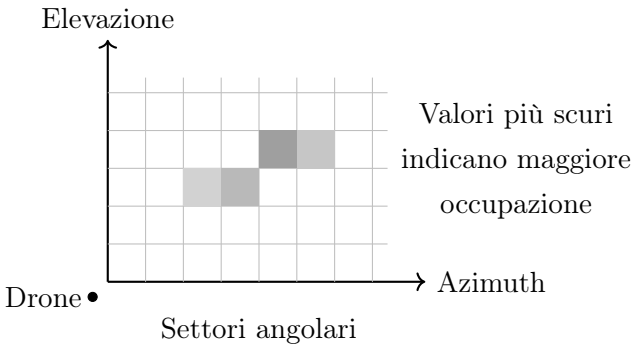


Figura 3.2. *Schema concettuale dell'istogramma sferico: lo spazio viene discretizzato secondo azimuth ed elevazione e ogni settore rappresenta il livello di occupazione in una direzione.*

3.3 Acquisizione dei dati dai sensori sonar

Il sistema utilizza cinque sonar simulati, posizionati in modo da coprire diverse aree intorno al drone. Il sonar frontale osserva la zona davanti al veicolo ed è il sensore principale per l'individuazione degli ostacoli lungo la direzione di avanzamento. I sonar laterali sinistro e destro permettono di valutare la presenza di ostacoli nelle direzioni laterali. Il sonar superiore e quello inferiore forniscono informazioni utili per la gestione della quota e per la percezione tridimensionale dell'ambiente.

I dati provenienti dai sonar vengono rappresentati come messaggi `PointCloud2`. Ogni messaggio contiene un insieme di punti tridimensionali, ciascuno dei quali descrive una possibile superficie rilevata dal sensore. Questi punti sono espressi tramite coordinate cartesiane e devono essere convertiti in una forma adatta alla costruzione dell'istogramma sferico.

Per ogni punto rilevato, il sistema calcola la distanza dal drone e la direzione corrispondente nello spazio. La distanza consente di stimare quanto un ostacolo sia vicino e quindi quanto debba influenzare la scelta del movimento. La direzione, espressa tramite azimuth ed elevazione, permette invece di determinare il settore dell'istogramma in cui inserire il contributo del punto.

La presenza di più sonar richiede una fase di trasformazione e fusione dei dati. I punti provenienti dai diversi sensori vengono riportati all'interno di una rappresentazione comune, in modo da costruire un'unica descrizione locale dell'ambiente. Questo permette al planner di ragionare su un quadro complessivo, invece di considerare separatamente ciascun sensore.

Un aspetto importante riguarda i limiti percettivi dei sensori simulati. In particolare, alcune superfici, come le pareti piatte della vasca, possono non essere rilevate correttamente dai sonar simulati. Per evitare che il drone si avvicini eccessivamente

alle pareti senza ricevere un segnale sensoriale adeguato, il sistema integra anche un calcolo geometrico della distanza dalle pareti. In questo modo, le pareti vengono considerate nella pianificazione anche quando non producono direttamente punti nella nuvola generata dai sonar.

3.4 Costruzione dell'istogramma sferico

La costruzione dell'istogramma sferico è una delle fasi centrali del metodo proposto. L'obiettivo è trasformare l'insieme dei punti rilevati dai sonar in una rappresentazione compatta dello spazio attorno al drone, nella quale ogni settore angolare indica il livello di rischio associato a una possibile direzione di movimento.

Per ogni punto della nuvola, il sistema calcola innanzitutto la distanza dal veicolo. Successivamente, il punto viene associato a una coppia di angoli: azimuth ed elevazione. Questi due valori determinano il settore della griglia sferica in cui il punto ricade. Una volta individuato il settore, il relativo valore di occupazione viene incrementato.

Il contributo di ciascun punto non è costante, ma dipende dalla sua distanza. Gli ostacoli più vicini rappresentano un pericolo maggiore rispetto a quelli lontani e devono quindi avere un peso più elevato nell'istogramma. Per questo motivo, il sistema utilizza un peso inversamente proporzionale al quadrato della distanza. Tale scelta rende il planner più sensibile agli ostacoli vicini, che sono quelli più rilevanti per la sicurezza immediata del drone.

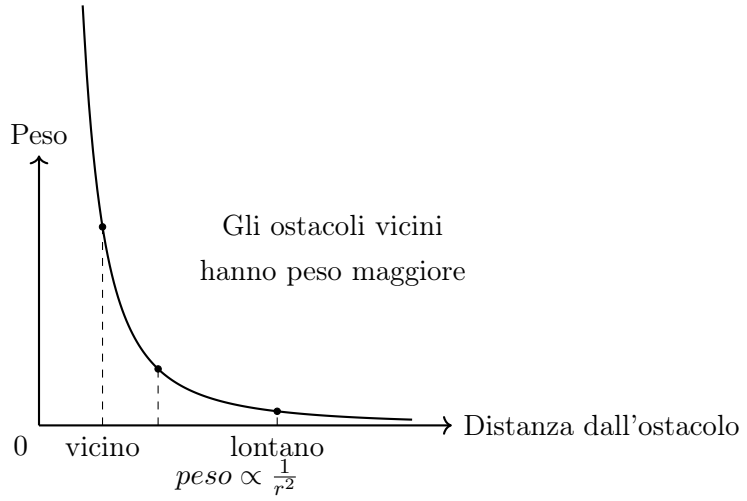


Figura 3.3. *Andamento qualitativo del peso assegnato agli ostacoli in funzione della distanza: ostacoli più vicini influenzano maggiormente l'istogramma.*

Oltre all'accumulo del peso, il sistema aggiorna per ogni settore la distanza minima osservata. Questo permette di conservare una misura diretta della vicinanza

dell'ostacolo più pericoloso in quella direzione. La combinazione tra peso accumulato e distanza minima fornisce una valutazione più completa del rischio rispetto all'utilizzo di una sola delle due informazioni.

Dopo la costruzione iniziale, l'istogramma viene sottoposto a una fase di smoothing. Lo smoothing riduce la sensibilità del sistema a piccoli rumori o variazioni isolate nei dati dei sensori. In pratica, il valore di un settore viene distribuito parzialmente anche ai settori vicini, rendendo più continua la rappresentazione dello spazio.

Lungo l'azimuth viene inoltre gestita la periodicità degli angoli. Infatti, la direzione corrispondente a 360 gradi è equivalente a quella corrispondente a 0 gradi. Per questo motivo, lo smoothing sull'azimuth deve considerare il collegamento tra il primo e l'ultimo settore della griglia. Questa gestione evita discontinuità artificiali nella rappresentazione dell'ambiente.

L'istogramma sferico ottenuto rappresenta quindi la base informativa su cui il planner effettua le proprie decisioni. I settori con valori elevati vengono interpretati come direzioni pericolose, mentre i settori con valori bassi e distanza minima sufficiente possono essere considerati candidati per il movimento del veicolo.

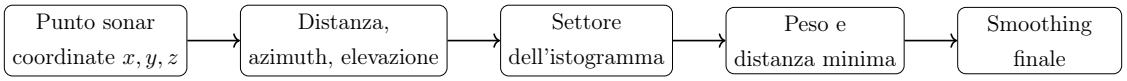


Figura 3.4. *Fasi principali della costruzione dell'istogramma sferico a partire dai punti rilevati dai sonar simulati.*

3.5 Fusione dei dati provenienti dai sensori

La presenza di più sonar consente al drone di ottenere una percezione più ampia dell'ambiente circostante, ma richiede anche una fase di fusione dei dati. Ciascun sensore osserva infatti una porzione diversa dello spazio e produce una propria nuvola di punti. Per permettere al planner di prendere decisioni coerenti, tali informazioni devono essere integrate in un'unica rappresentazione comune.

Nel sistema sviluppato, i dati provenienti dai sonar vengono trasformati e accumulati all'interno dello stesso istogramma sferico tridimensionale. Ogni punto rilevato contribuisce al settore corrispondente in base alla sua direzione e alla sua distanza dal drone. In questo modo, il planner non considera i sensori come sorgenti separate, ma costruisce una descrizione complessiva dello spazio locale.

Il sonar frontale fornisce le informazioni principali per il movimento in avanti, poiché rileva gli ostacoli posti direttamente lungo la traiettoria del veicolo. I sonar laterali permettono invece di valutare quali direzioni siano più adatte per una ma-

novra evasiva orizzontale. Il sonar superiore e quello inferiore contribuiscono alla valutazione dell'ambiente sopra e sotto il drone e supportano il controllo della quota.

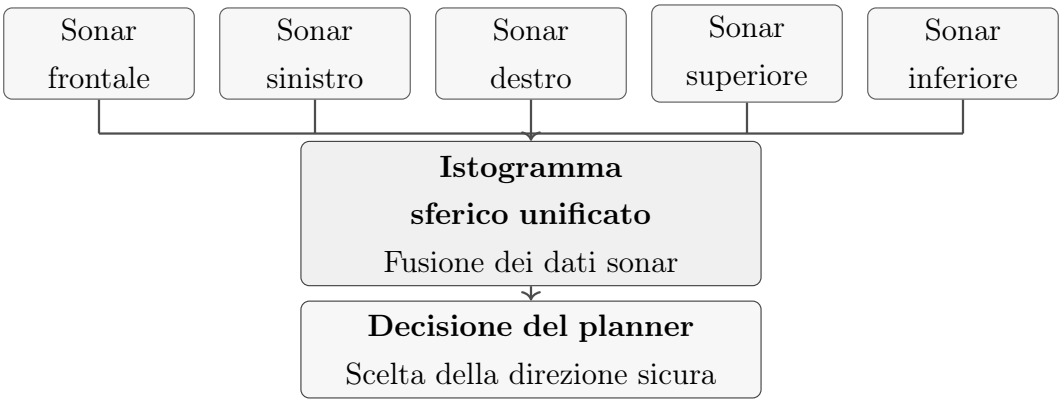


Figura 3.5. *Fusione dei dati provenienti dai diversi sonar in una singola rappresentazione tridimensionale locale.*

La fusione dei dati sensoriali è particolarmente importante negli scenari tridimensionali. Un ostacolo potrebbe infatti essere percepito dal sonar frontale e, contemporaneamente, dai sonar laterali o verticali. Integrare queste informazioni in un'unica struttura permette al sistema di avere una rappresentazione più coerente dell'ambiente.

Nel sistema viene inoltre mantenuta una memoria temporanea di alcuni ostacoli rilevati. Questa memoria non costituisce una mappa globale dell'ambiente, ma permette di conservare per un breve periodo informazioni utili in condizioni particolari, ad esempio durante manovre di retromarcia o recupero. In questo modo, il sistema può evitare di considerare improvvisamente libero uno spazio in cui era stato appena rilevato un ostacolo.

L'approccio adottato permette quindi di trasformare più sorgenti sensoriali in una singola struttura decisionale. L'istogramma sferico diventa il punto di raccolta delle informazioni percettive e costituisce la base per la successiva scelta della direzione di movimento.

3.6 Ricerca della direzione sicura

Una volta costruito l'istogramma sferico, il planner deve individuare una direzione di movimento che consenta al drone di proseguire riducendo il rischio di collisione. Questa fase rappresenta il cuore del sistema di *collision avoidance*, poiché trasforma la rappresentazione dell'ambiente in una decisione di navigazione.

La ricerca della direzione sicura non consiste semplicemente nello scegliere il settore con valore minimo nell'istogramma. Una direzione apparentemente libera potrebbe infatti richiedere una manovra troppo brusca, essere troppo vicina a un ostacolo oppure portare il drone verso una parete. Per questo motivo, il planner valuta i settori candidati considerando diversi fattori: il livello di occupazione, la distanza minima dagli ostacoli, la direzione corrente del veicolo, la direzione dell'obiettivo e la realizzabilità della manovra.

In condizioni normali, il sistema tende a preferire direzioni che permettono al drone di mantenere un movimento regolare verso il target. Quando la zona frontale è libera, il veicolo può procedere in modalità di crociera. Quando invece viene rilevato un ostacolo lungo il corridoio di avanzamento, il planner cerca una via di fuga tra le direzioni disponibili.

Per rendere la scelta più robusta, il sistema utilizza una valutazione del corridoio davanti al drone. Invece di considerare soltanto un singolo settore angolare, il planner valuta se esiste uno spazio percorribile lungo una certa direzione per una distanza sufficiente. Questo approccio riduce il rischio di scegliere direzioni che appaiono libere localmente ma che risultano impraticabili poco più avanti.

Quando viene individuata una direzione sicura, il sistema può generare un waypoint locale. Il waypoint rappresenta un punto intermedio verso cui il drone si dirige durante la manovra evasiva. L'utilizzo di waypoint locali evita che la direzione venga ricreata completamente a ogni ciclo di controllo, riducendo oscillazioni e cambiamenti improvvisi di traiettoria.

Nelle campagne sperimentali attuali, la manovra evasiva principale è basata sulla scelta di corridoi percorribili e waypoint locali nello spazio circostante. La componente verticale viene comunque considerata nella percezione e nel controllo della quota, ma la fuga verticale esplicita rimane disattivata nei mondi attuali. Questa scelta consente di mantenere più stabile il comportamento del veicolo durante i test e di concentrare la valutazione sull'efficacia del planner locale rispetto alla densità degli ostacoli.

Un altro elemento considerato dal planner è la distanza dalle pareti della vasca. Poiché alcune pareti piatte non vengono rilevate in modo affidabile dai sonar simulati, il sistema integra un controllo geometrico della distanza dalle pareti. In

questo modo, una direzione che sembrerebbe libera sulla base del solo istogramma può essere scartata se porta il veicolo troppo vicino ai limiti dell'ambiente.

Il risultato finale della ricerca è una direzione desiderata, espressa principalmente in termini di orientamento orizzontale e, quando necessario, di correzione della quota. Tale direzione viene poi passata al modulo di controllo, che si occupa di trasformarla in comandi per gli attuatori del drone.

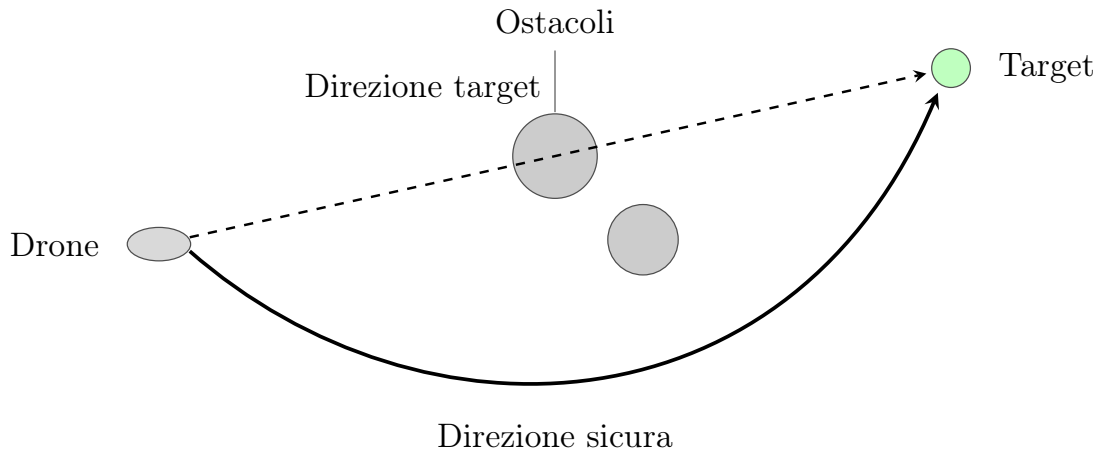


Figura 3.6. *Schema concettuale della ricerca di una direzione sicura: il planner evita la direzione verso il target quando il corridoio è ostruito e seleziona una traiettoria locale alternativa.*

3.7 Controllo del veicolo subacqueo

Dopo aver selezionato una direzione sicura, il sistema deve trasformare tale decisione in comandi fisici per il veicolo. Nel caso del drone subacqueo Tethys, il controllo avviene tramite tre elementi principali: pinne verticali, pinne orizzontali e propulsore.

Le pinne verticali vengono utilizzate per modificare la direzione orizzontale del drone. Agendo su di esse, il sistema può generare una variazione dello yaw, permettendo al veicolo di sterzare verso sinistra o verso destra. Questo controllo è fondamentale per le manovre evasive laterali e per mantenere una traiettoria compatibile con la direzione scelta dal planner.

Le pinne orizzontali influenzano invece il pitch e la profondità del veicolo. Attraverso questi attuatori, il drone può correggere la propria quota, mantenersi entro i limiti di profondità e compensare eventuali variazioni indesiderate. Anche se la fuga verticale esplicita non è utilizzata nei mondi sperimentali attuali, il controllo della quota rimane una componente essenziale per la stabilità del veicolo.

Il propulsore fornisce la spinta necessaria all'avanzamento del drone. In condizioni normali, esso mantiene il movimento in avanti; in situazioni più critiche, la

spinta può essere ridotta per rallentare il veicolo o modificata per eseguire una manovra di emergenza. La regolazione della spinta contribuisce quindi alla sicurezza complessiva del sistema, soprattutto quando un ostacolo viene rilevato a distanza ravvicinata.

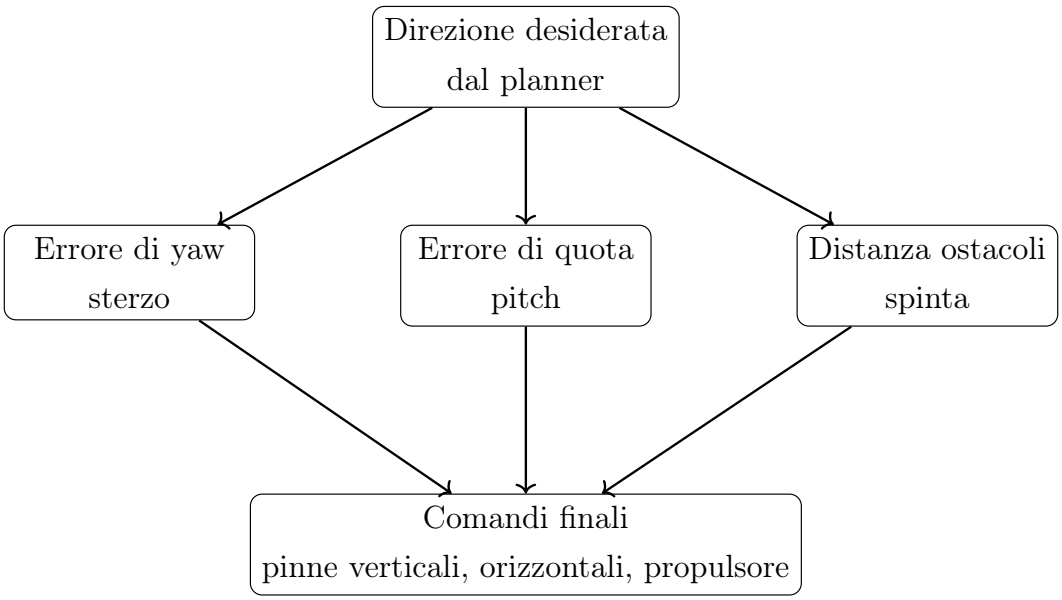


Figura 3.7. *Schema del passaggio dalla direzione scelta dal planner ai comandi inviati agli attuatori del veicolo.*

Il controllo del veicolo deve tenere conto anche delle caratteristiche dinamiche dell'ambiente subacqueo. A differenza di un robot terrestre, un AUV è soggetto a effetti idrodinamici e a fenomeni di smorzamento che influenzano la risposta ai comandi. Ad esempio, una variazione delle pinne orizzontali non produce immediatamente un cambio di quota, ma genera una risposta progressiva del veicolo. Per questo motivo, le correzioni verticali devono essere controllate con attenzione.

Nel sistema, il comando finale può essere interpretato come la combinazione di tre grandezze: spinta del propulsore, comando di sterzo orizzontale e comando verticale. Questi valori vengono calcolati a partire dallo stato del veicolo, dalla direzione selezionata dal planner, dalla quota desiderata e dalla distanza dagli ostacoli.

Il modulo di controllo, quindi, non si limita a inviare direttamente la direzione scelta dal planner, ma tiene conto della stabilità del movimento, della quota del veicolo e della necessità di evitare comandi troppo bruschi. L'obiettivo è ottenere un comportamento sufficientemente regolare e sicuro all'interno della simulazione.

3.8 Gestione degli stati di navigazione

Per rendere il comportamento del drone più robusto, il sistema utilizza una logica basata su stati di navigazione. Ogni stato rappresenta una condizione operativa diversa e determina il modo in cui il veicolo deve reagire all'ambiente circostante.

Lo stato principale è la modalità `CRUISE`, utilizzata durante la navigazione normale. In questa fase, il drone procede cercando di mantenere una traiettoria sicura verso il target. Il planner continua a elaborare i dati provenienti dai sonar e a controllare che non siano presenti ostacoli pericolosi lungo la direzione di movimento. Se l'ambiente risulta sufficientemente libero, il veicolo mantiene il comportamento di crociera.

Quando viene rilevato un ostacolo che richiede una deviazione, il sistema può passare allo stato `ESCAPE`. In questa modalità, il drone esegue una manovra evasiva. La direzione di fuga viene scelta sulla base dell'istogramma sferico, della valutazione dei corridoi e della direzione del target. Durante questa fase, il sistema tende a mantenere temporaneamente una direzione di evasione stabile, evitando cambiamenti continui che potrebbero causare oscillazioni o comportamenti incerti.

Nei casi più critici, quando un ostacolo viene rilevato a distanza molto ridotta o quando il corridoio davanti al drone non è sicuro, il sistema può entrare nello stato `BRAKE`. Questo stato rappresenta una condizione di emergenza nella quale il drone riduce il movimento in avanti o esegue una retromarcia controllata per diminuire il rischio di collisione immediata.

Accanto a questi tre stati principali, il sistema utilizza alcune logiche di protezione aggiuntive. Tra queste rientrano il recupero della quota, il controllo dei limiti di profondità, la stabilizzazione del veicolo e la fase di avvicinamento finale al target. Tali logiche non rappresentano necessariamente stati separati della macchina principale, ma possono intervenire per modificare temporaneamente il comportamento del drone quando vengono rilevate condizioni particolari.

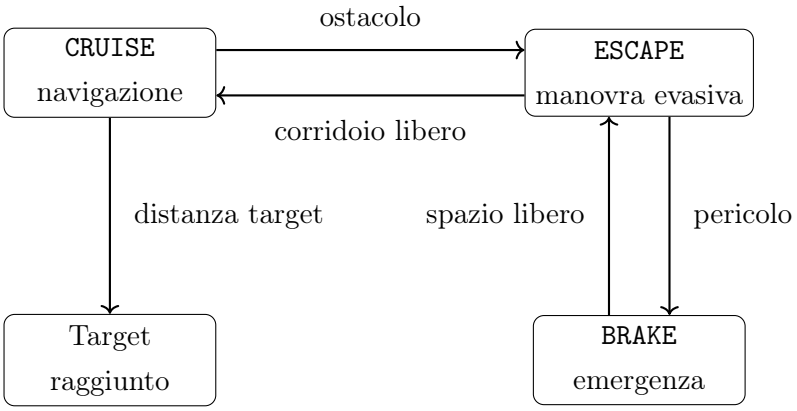


Figura 3.8. *Schema semplificato della macchina a stati utilizzata per organizzare il comportamento del drone durante la navigazione, integrata da logiche di protezione della quota, stabilizzazione e avvicinamento finale al target.*

La transizione tra gli stati dipende principalmente dalla distanza dagli ostacoli, dalla configurazione dell'ambiente, dalla possibilità di individuare una direzione sicura e dalla posizione del target. Se il drone si trova in condizioni normali, rimane in `CRUISE`; se un ostacolo viene rilevato a distanza significativa ma gestibile, passa a `ESCAPE`; se invece il pericolo è immediato, passa a `BRAKE`.

Durante lo sviluppo sono emerse alcune criticità legate alla gestione degli stati. In particolare, lo stato di emergenza può generare comportamenti indesiderati se il drone rimane bloccato in retromarcia o se la manovra lo porta verso un'altra zona pericolosa. Per questo motivo, sono stati introdotti meccanismi di controllo aggiuntivi, come timer per rilevare condizioni di blocco, strategie di uscita forzata e gestione separata di casi particolari, ad esempio la vicinanza al fondale o alle pareti.

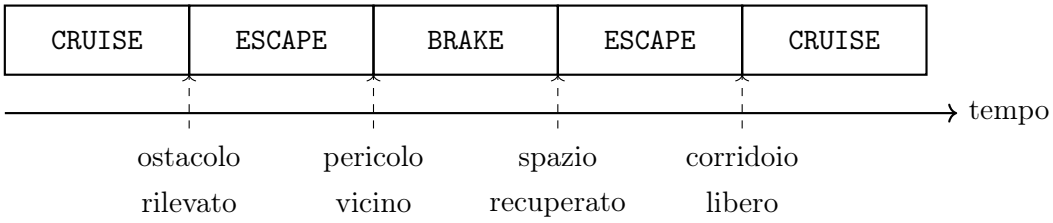


Figura 3.9. *Esempio qualitativo di evoluzione temporale degli stati durante una manovra di evitamento ostacolo.*

La macchina a stati consente quindi di organizzare il comportamento del drone in modo più chiaro e controllabile. Essa permette di distinguere tra navigazione ordinaria, manovra evasiva e gestione dell'emergenza, migliorando la robustezza complessiva del sistema rispetto a un approccio basato su un'unica regola di controllo.

Nel complesso, il metodo combina percezione locale, istogramma sferico tridimen-

sionale, fusione dei dati sonar, ricerca di direzioni sicure, waypoint locali, controllo della quota e gestione degli stati. Questa architettura permette al drone subacqueo di reagire alla presenza di ostacoli in ambiente tridimensionale, mantenendo un comportamento coerente con i vincoli dinamici del veicolo e con le condizioni operative della simulazione.

Capitolo 4

Implementation

In questo capitolo viene descritta l'implementazione del sistema di *collision avoidance* tridimensionale sviluppato durante il tirocinio. Dopo aver presentato nel capitolo precedente la metodologia generale, si passa ora agli aspetti pratici: organizzazione del workspace ROS 2, struttura del pacchetto, nodo principale, topic utilizzati, comunicazione con Gazebo, modello del drone Tethys, script di avvio e raccolta dei dati sperimentali.

L'implementazione è stata realizzata in C++ all'interno di un workspace ROS 2. Il sistema è stato testato in ambiente Ubuntu con ROS 2 Humble e Ignition Gazebo Fortress. Il codice sviluppato si integra con i pacchetti relativi al modello Tethys LRAUV e con i plugin Gazebo necessari alla simulazione del veicolo, dei sensori e degli attuatori.

L'obiettivo dell'implementazione non è stato soltanto realizzare un algoritmo teorico di evitamento ostacoli, ma inserirlo in una simulazione funzionante, nella quale il drone riceve dati dai sonar, calcola una direzione sicura, invia comandi agli attuatori e produce metriche utili alla valutazione sperimentale.

4.1 Struttura del workspace ROS 2

Il progetto è stato organizzato all'interno di un workspace ROS 2 denominato `ros2_tethys`. Un workspace ROS 2 è una directory contenente uno o più pacchetti software, che possono essere compilati insieme tramite lo strumento `colcon`. In questo caso, il workspace contiene sia i pacchetti relativi al modello del drone Tethys sia il pacchetto sviluppato per la *collision avoidance*.

La struttura generale può essere schematizzata come segue:

```
ros2_tethys/
|-- src/
|   |-- collision_avoidance/
|   |-- lrauv_description/
|   |-- lrauv_gazebo_plugins/
|   `-- lrauv_system_tests/
|-- scripts/
|-- worlds/
`-- docs/
```

La directory `src` contiene i pacchetti ROS 2. Il pacchetto principale sviluppato durante il tirocinio è `collision_avoidance`, che implementa il nodo di controllo e il planner locale. I pacchetti `lrauv_description` e `lrauv_gazebo_plugins` forniscono rispettivamente la descrizione del veicolo Tethys e i plugin necessari alla simulazione. Il pacchetto `lrauv_system_tests` contiene invece strumenti di test e integrazione.

La directory `scripts` contiene gli script utilizzati per semplificare l'avvio della simulazione, il bridge tra ROS 2 e Gazebo, la verifica dei mondi e l'esecuzione delle campagne sperimentali. La directory `worlds` contiene i sei mondi definitivi, tre per lo scenario a un drone e tre per quello a cinque droni. La directory `docs` raccoglie i documenti di supporto e i file CSV prodotti dagli avvii manuali.

La compilazione del pacchetto principale avviene tramite il comando:

```
source /opt/ros/humble/setup.bash
colcon build --packages-select collision_avoidance
```

Dopo la compilazione, il workspace deve essere caricato nell'ambiente della shell tramite il file di setup generato:

```
source install/setup.bash
```

Questa operazione permette a ROS 2 di riconoscere i pacchetti compilati e di eseguire il nodo sviluppato.

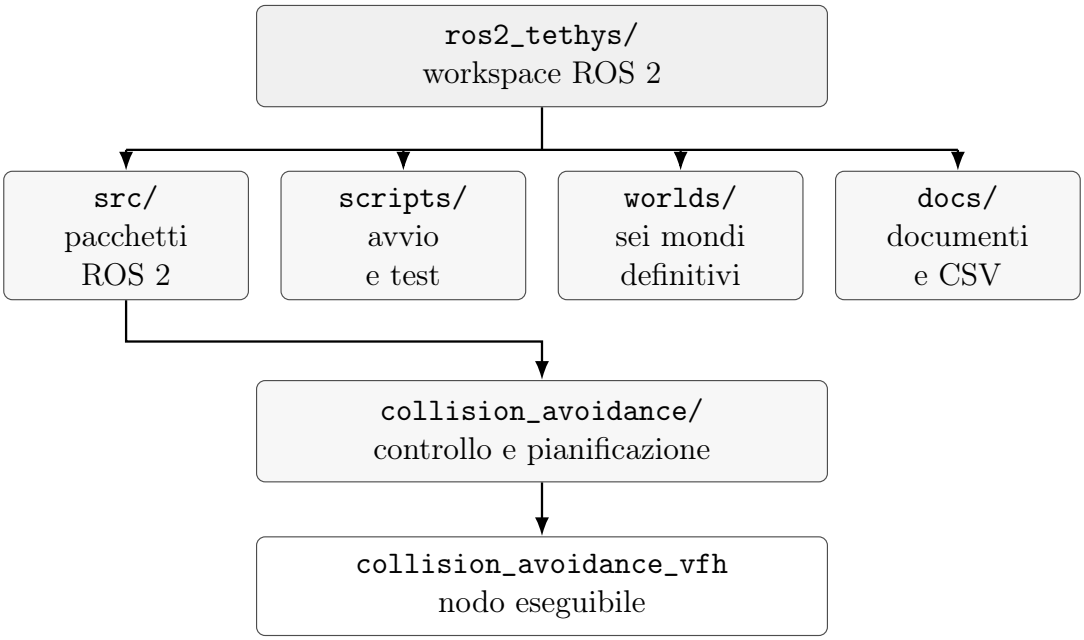


Figura 4.1. *Organizzazione generale del workspace ROS 2 utilizzato nel progetto.*

4.2 Pacchetto `collision_avoidance`

Il pacchetto `collision_avoidance` rappresenta il componente principale sviluppato durante il tirocinio. Esso contiene il codice responsabile della percezione locale, della costruzione dell'istogramma sferico, della scelta della direzione sicura, della gestione degli stati di navigazione e della generazione dei comandi di controllo.

Il pacchetto è stato implementato in C++ e segue la struttura tipica di un pacchetto ROS 2 basato su `ament_cmake`. I file principali sono:

- `package.xml`, che descrive il pacchetto e le sue dipendenze;
- `CMakeLists.txt`, che definisce le regole di compilazione;
- `collision_avoidance_node.hpp`, che dichiara la classe, i metodi e le variabili membro;
- `collision_avoidance_node.cpp`, che contiene il costruttore e `control_loop()`, coordinando il ciclo di controllo e la macchina a stati;
- `main.cpp`, che inizializza ROS 2 e avvia il nodo;
- `sensors.cpp`, che gestisce sonar, fusione dei dati e posa;
- `navigation.cpp`, che gestisce target, direzione preferita e waypoint;

- `motion_control.cpp`, che gestisce pinne, propulsione, quota e limiti di sicurezza;
- `metrics.cpp`, che pubblica le metriche e scrive i CSV;
- `vfh_planner.cpp`, che contiene le funzioni geometriche del planner;
- `sph_hist.hpp`, che definisce l'istogramma sferico e le trasformazioni dei sonar;
- `vfh_planner.hpp`, che contiene le costanti operative e l'interfaccia del planner;
- `motion_safety.hpp`, dedicato ad alcune funzioni di sicurezza legate alla frenata e alla retromarcia;
- `depth_controller.hpp`, utilizzato per il controllo della quota;
- `obstacle_memory.hpp`, che implementa una memoria temporanea degli ostacoli;
- `vehicle_state.hpp`, che rappresenta lo stato cinematico del veicolo.

Il pacchetto dipende da alcune librerie ROS 2 fondamentali. In particolare, utilizza `rclcpp` per la realizzazione del nodo, `sensor_msgs` per la gestione delle nuvole di punti, `std_msgs` per la pubblicazione dei comandi numerici e `tf2_msgs` per la gestione delle informazioni di posa provenienti dalla simulazione.

La suddivisione del nodo separa le responsabilità di supporto senza modificare la logica della macchina a stati. Il file principale coordina il ciclo di controllo, mentre acquisizione dei sensori, navigazione, attuazione e metriche sono implementate nei moduli dedicati. La funzione `control_loop()` conserva al suo interno l'ordine completo delle decisioni di frenata, quota, `ESCAPE`, waypoint e stabilizzazione. Il planner VFH rimane il modulo dedicato alla selezione della direzione sicura sulla base dell'istogramma, dei corridoi percorribili e delle condizioni dell'ambiente.

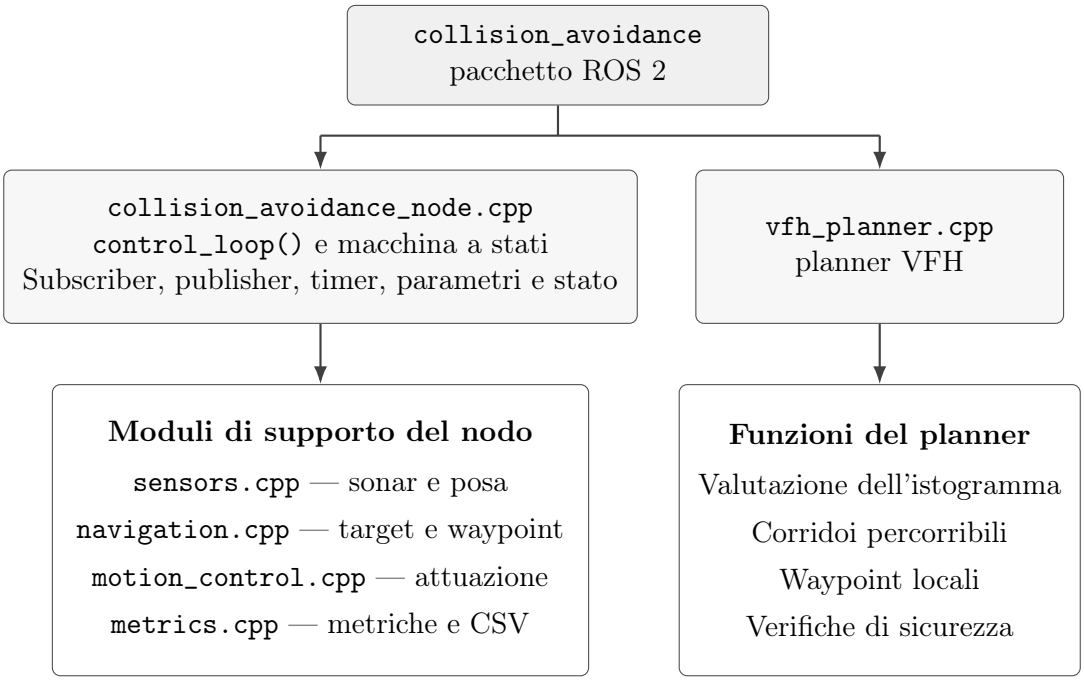


Figura 4.2. *Struttura logica del pacchetto* `collision_avoidance`.

4.3 Nodo principale `collision_avoidance_vfh`

Il nodo principale del sistema viene eseguito con il nome `collision_avoidance_vfh`. Esso rappresenta il collegamento tra l'algoritmo di *collision avoidance* e l'ambiente ROS 2/Gazebo. Il nodo gestisce sia la ricezione dei dati sensoriali sia la pubblicazione dei comandi verso gli attuatori simulati.

Il nodo è organizzato attorno a un ciclo di controllo periodico eseguito a 5 Hz in tempo simulato. A ogni iterazione, il sistema verifica la disponibilità dei dati provenienti dai sonar, aggiorna la rappresentazione locale dell'ambiente, valuta lo stato del drone e calcola i comandi da inviare agli attuatori.

Le principali responsabilità del nodo sono:

- sottoscrivere i topic dei sonar simulati;
- ricevere la posa del drone all'interno del mondo simulato;
- ricevere gli eventuali contatti fisici usati per le metriche;
- fondere i dati sensoriali in un istogramma sferico;
- invocare il planner locale per selezionare una direzione sicura;
- gestire la macchina a stati di navigazione;

- pubblicare i comandi per pinne verticali, pinne orizzontali e propulsore;
- registrare metriche sperimentali durante le prove.

Il nodo utilizza diversi parametri configurabili. Tra questi rientrano il nome del drone, il nome del mondo di simulazione, la posizione del target, il raggio di arrivo al goal, i limiti di profondità, il numero di ostacoli e il percorso del file CSV utilizzato per salvare i risultati sperimentali. L'utilizzo di parametri rende il sistema più flessibile, poiché permette di eseguire lo stesso nodo in scenari diversi senza modificare direttamente il codice sorgente.

Una caratteristica importante dell'implementazione è la possibilità di eseguire più istanze del nodo, una per ciascun drone. Nei test multi-drone, ogni veicolo utilizza un proprio nodo di controllo, con topic e parametri associati al relativo modello. Non è stata implementata una coordinazione esplicita tra droni: ciascun drone esegue il proprio algoritmo di *collision avoidance* sulla base delle informazioni sensoriali disponibili e tratta gli altri veicoli come ostacoli rilevati dai sonar.

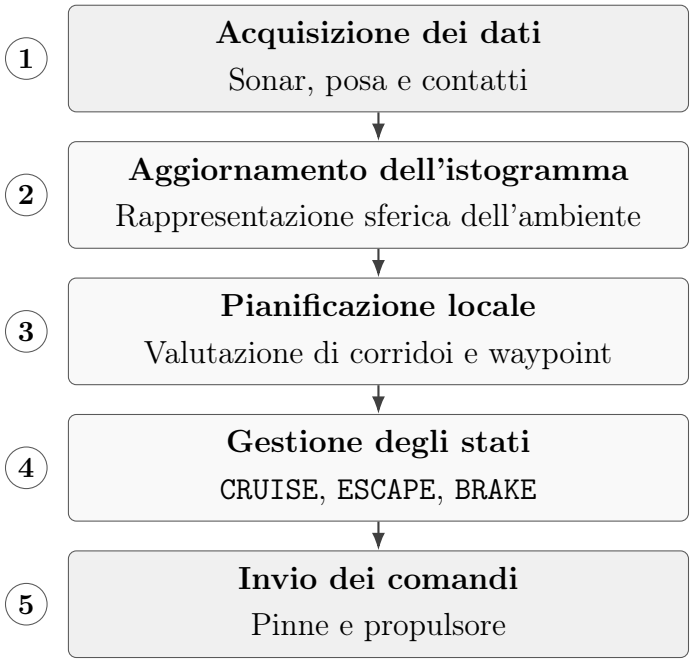


Figura 4.3. *Flusso interno semplificato del nodo principale di* collision avoidance.

4.4 Comunicazione tra ROS 2 e Gazebo

La simulazione del drone e dell'ambiente avviene in Ignition Gazebo, mentre il nodo di controllo è eseguito in ROS 2. Per consentire lo scambio di dati tra questi due sistemi è necessario utilizzare un bridge. Nel progetto è stato utilizzato `ros_gz_bridge`, che permette di convertire messaggi Gazebo in messaggi ROS 2 e viceversa.

Il bridge viene utilizzato per rendere disponibili a ROS 2 i dati generati dai sensori simulati. In particolare, le nuvole di punti prodotte dai sonar vengono convertite in messaggi `sensor_msgs/msg/PointCloud2`. Allo stesso modo, i comandi pubblicati dal nodo ROS 2 vengono inoltrati ai controller degli attuatori presenti nella simulazione Gazebo.

La comunicazione può essere schematizzata in due flussi principali:

- flusso sensoriale, da Gazebo verso ROS 2;
- flusso di controllo, da ROS 2 verso Gazebo.

Nel flusso sensoriale, Gazebo simula i sonar e produce dati relativi agli ostacoli rilevati. Tali dati vengono convertiti dal bridge e ricevuti dal nodo di *collision avoidance*. Nel flusso di controllo, il nodo ROS 2 calcola i comandi e li pubblica sui topic degli attuatori; il bridge li trasferisce a Gazebo, che aggiorna il comportamento fisico del drone.

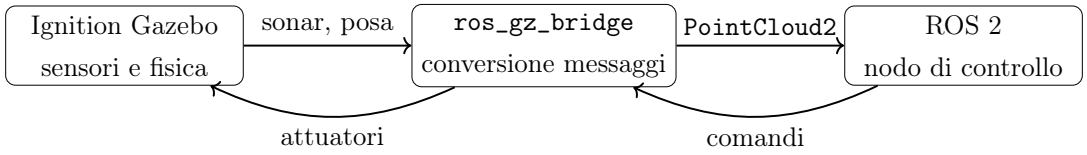


Figura 4.4. *Comunicazione bidirezionale tra Gazebo e ROS 2 tramite* `ros_gz_bridge`.

Questa architettura permette di mantenere separati il simulatore e il software di controllo. Il nodo ROS 2 non deve gestire direttamente la fisica del veicolo, mentre Gazebo non contiene la logica decisionale dell'algoritmo. I due sistemi comunicano attraverso topic ben definiti.

4.5 Topic ROS 2 utilizzati

Il sistema utilizza diversi topic ROS 2 per ricevere dati dalla simulazione e inviare comandi al drone. I topic sono parametrizzati in base al nome del modello, in modo da poter utilizzare lo stesso codice sia per un singolo drone sia per scenari multi-drone.

I principali topic di ingresso sono quelli relativi ai sonar simulati. Ogni sonar pubblica una nuvola di punti, ricevuta dal nodo come messaggio `PointCloud2`. Il nodo riceve inoltre la posa dei modelli e i contatti fisici, utilizzati per la raccolta delle metriche sperimentali.

Topic	**Tipo**	**Ruolo**
`/model/<n>/frontal_sonar/points`	`PointCloud2`	Dati del sonar frontale
`/model/<n>/left_sonar/points`	`PointCloud2`	Dati del sonar sinistro
`/model/<n>/right_sonar/points`	`PointCloud2`	Dati del sonar destro
`/model/<n>/bottom_sonar/points`	`PointCloud2`	Dati del sonar inferiore
`/model/<n>/top_sonar/points`	`PointCloud2`	Dati del sonar superiore
`/world/buoyant_tethys/dynamic_pose/info`	`TFMessage`	Informazioni sulla posa dei modelli
`/model/<n>/contacts/<link>`	`Contacts`	Contatti fisici usati per le metriche

Tabella 4.1. *Principali topic ROS 2 di ingresso utilizzati dal nodo di* collision avoidance.

I topic di uscita sono utilizzati per controllare le pinne verticali, le pinne orizzontali e il propulsore. Il bridge si occupa poi di trasferire questi comandi verso i topic Gazebo effettivamente ascoltati dai plugin del modello.

Topic	**Tipo**	**Ruolo**
`/model/<n>/vertical_fins/cmd_pos`	`Float64`	Comando per le pinne verticali
`/model/<n>/horizontal_fins/cmd_pos`	`Float64`	Comando per le pinne orizzontali
`/model/<n>/joint/propeller_joint/cmd_thrust`	`Float64`	Comando di spinta del propulsore
`/ca_metrics`	`String`	Metriche periodiche del sistema

Tabella 4.2. *Principali topic ROS 2 di uscita utilizzati dal nodo di* collision avoidance.

La separazione tra topic di ingresso e topic di uscita rende chiaro il flusso dei dati. Il nodo riceve informazioni sull'ambiente e sullo stato del drone, elabora tali informazioni e pubblica comandi verso gli attuatori. Questa struttura segue il paradigma tipico dei sistemi robotici basati su ROS 2.

4.6 Modello del drone subacqueo Tethys

Il veicolo utilizzato nel progetto è il modello simulato Tethys LRAUV. Il modello include la descrizione geometrica del drone, i link fisici, i giunti, i sensori e i plugin necessari a simulare il comportamento in ambiente subacqueo.

Nel contesto del progetto, il modello Tethys è stato utilizzato come piattaforma sulla quale integrare il sistema di *collision avoidance*. Il lavoro non ha riguardato la progettazione meccanica del veicolo, ma la configurazione e l'utilizzo del modello esistente all'interno di Gazebo.

Nei mondi utilizzati dal progetto è stato impiegato un modello in forma *flat*, cioè con il contenuto del modello Tethys incorporato direttamente nel file del mondo simulato. In questo modo, i plugin principali del veicolo, come thruster, controllori delle pinne, lift/drag, idrodinamica e galleggiamento, risultano presenti a livello del modello istanziato nel mondo.

Gli elementi principali del modello rilevanti per questo lavoro sono:

- il corpo principale del veicolo;
- le pinne verticali, usate per il controllo della direzione orizzontale;
- le pinne orizzontali, usate per il controllo del pitch e della quota;
- il giunto del propulsore, usato per generare la spinta;
- i cinque sonar simulati;
- i plugin idrodinamici e di controllo.

Una particolarità del modello è che il veicolo avanza lungo l'asse body $-X$. Per questo motivo, negli script viene utilizzato un parametro di compensazione dell'orientamento, indicato come `yaw_bias`, impostato a π. Tale scelta consente al nodo di controllo di interpretare correttamente la direzione di avanzamento del veicolo rispetto al sistema di riferimento del mondo.

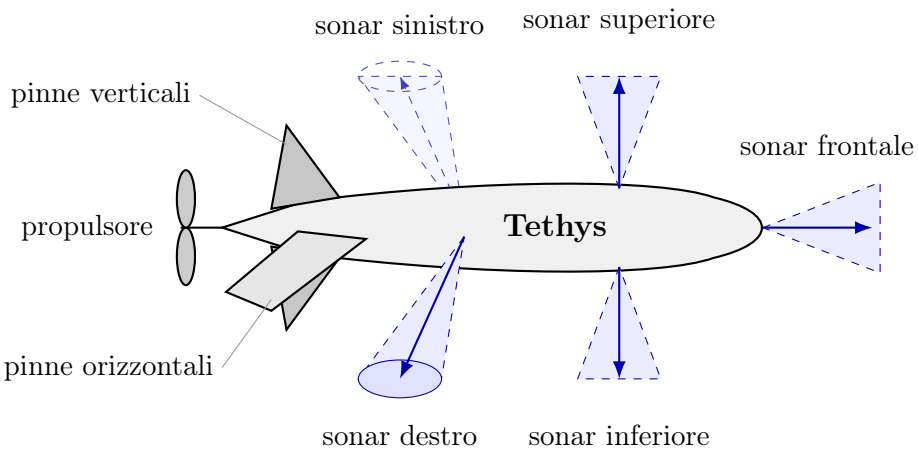


Figura 4.5. *Schema semplificato del modello Tethys e dei principali elementi utilizzati dal sistema di controllo.*

La risposta del veicolo ai comandi non è istantanea. Il comportamento è influenzato dai plugin fisici e idrodinamici della simulazione, che introducono inerzia, smorzamento e dinamiche tipiche dell'ambiente subacqueo. Per questo motivo, durante lo sviluppo è stato necessario calibrare il controllo in modo da evitare oscillazioni, manovre troppo brusche o situazioni di blocco.

4.7 Configurazione dei sonar simulati

Il sistema utilizza cinque sonar simulati tramite sensori `gpu_lidar`. Ogni sonar osserva una regione diversa dello spazio e pubblica una nuvola di punti. I dati vengono poi trasformati nel riferimento del veicolo e inseriti nell'istogramma sferico.

La configurazione dei sonar è riportata nella Tabella 4.3. Il sonar frontale ha il ruolo principale nell'individuazione degli ostacoli lungo la direzione di avanzamento, mentre i sonar laterali e verticali ampliano la percezione dell'ambiente circostante.

Sonar	**Campioni**	**Frequenza**	**Portata**
Frontale	30×15	5 Hz	35 m
Sinistro	10×5	2 Hz	10 m
Destro	10×5	2 Hz	10 m
Inferiore	1 raggio	2 Hz	10 m
Superiore	5×5	2 Hz	10 m

Tabella 4.3. *Configurazione dei sonar simulati utilizzati dal sistema.*

Una criticità emersa durante lo sviluppo riguarda il fatto che le pareti piatte della vasca non vengono rilevate in modo affidabile dai sensori `gpu_lidar`. Per evitare che

il drone si avvicini eccessivamente ai limiti del mondo simulato, il sistema integra un calcolo analitico della distanza dalle pareti, utilizzato durante la valutazione dei corridoi percorribili.

4.8 Configurazione dei mondi simulati

Gli scenari di test sono definiti tramite file SDF. Ogni mondo contiene la vasca, il modello Tethys, gli ostacoli, i sensori e i plugin necessari alla simulazione. La configurazione dei mondi è stata progettata per valutare il comportamento del sistema in condizioni di difficoltà crescente.

La campagna sperimentale finale è organizzata in due scenari principali:

- scenario con un singolo drone;
- scenario con cinque droni.

Per ciascuno scenario sono previsti tre livelli di difficoltà. Nel caso del singolo drone, il numero di ostacoli aumenta di cinque unità per livello. Nel caso dello scenario con cinque droni, il numero di ostacoli aumenta di dieci unità per livello. Il resto della configurazione viene mantenuto il più possibile invariato all'interno dello stesso scenario, in modo da isolare l'effetto della densità di ostacoli.

Scenario	**Livello 1**	**Livello 2**	**Livello 3**
1 drone	30 ostacoli	35 ostacoli	40 ostacoli
5 droni	40 ostacoli	50 ostacoli	60 ostacoli

Tabella 4.4. *Organizzazione della campagna sperimentale finale in scenari e livelli di difficoltà.*

Gli ostacoli sono modellati principalmente come pilastri cilindrici verticali. Questa scelta consente di costruire scenari ripetibili e sufficientemente complessi per testare l'algoritmo di evitamento ostacoli. La presenza di ostacoli statici permette di osservare il comportamento del drone durante l'avvicinamento, la deviazione e il ritorno verso la direzione del target.

Negli scenari multi-drone, ogni veicolo parte da una posizione iniziale differente e dispone di un proprio target. I droni non condividono una pianificazione centralizzata: ciascuno utilizza la propria istanza del nodo di *collision avoidance*. Questo permette di valutare il comportamento del sistema in presenza di più agenti, pur mantenendo una logica di controllo locale.

Lo scenario a tre droni non viene utilizzato nella valutazione finale. Esso è stato considerato in una fase precedente dello sviluppo, ma è stato scartato perché non rientra nella campagna sperimentale definitiva e non viene impiegato nelle statistiche finali.

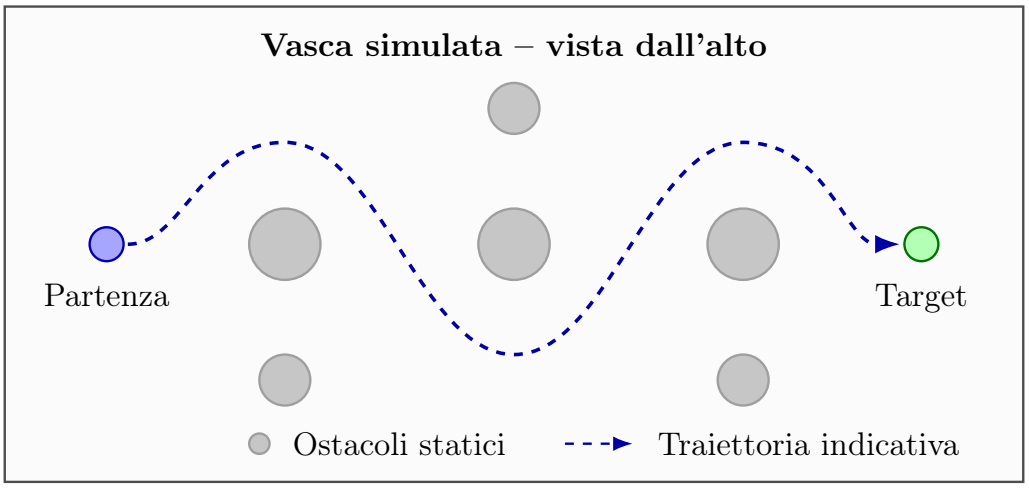


Figura 4.6. *Schema concettuale di uno scenario simulato: la traiettoria indicativa aggira gli ostacoli posti lungo la direzione verso il target finale.*

4.9 Script di avvio della simulazione

Per semplificare l'esecuzione del sistema sono stati utilizzati script di avvio. Gli script permettono di configurare l'ambiente, avviare Gazebo, attivare il bridge tra Gazebo e ROS 2 ed eseguire il nodo di *collision avoidance*.

Una tipica sessione di test richiede diverse operazioni:

1. compilazione del workspace;
2. caricamento dell'ambiente ROS 2;
3. avvio del mondo Gazebo;
4. avvio del bridge dei topic;
5. esecuzione del nodo di controllo;
6. raccolta delle metriche sperimentali.

Senza script, la simulazione viene avviata manualmente tramite tre terminali separati: uno per Gazebo, uno per il bridge e uno per il nodo di controllo. Gli script permettono invece di ridurre il rischio di errori e di ripetere più facilmente gli esperimenti.

Per le campagne sperimentali sono stati preparati anche comandi eseguibili direttamente dal terminale Ubuntu/Linux. Questi comandi avviano automaticamente le prove dello stesso mondo in sequenza, senza richiedere l'intervento manuale tra una prova e la successiva. L'esecuzione prosegue fino al raggiungimento di dieci successi, così da ottenere per ogni scenario il numero di risultati previsto dalla campagna sperimentale.

Un esempio di avvio di una campagna con cinque droni, mediante lo script presente nel progetto, è il seguente:

```
cd ~/ros2_tethys
source /opt/ros/humble/setup.bash
source install/setup.bash

MONDO=mondoMulti5.sdf bash scripts/prove_multi.sh \
  5 201 2000 10 5
```

Il comando entra nel workspace e carica l'ambiente ROS 2 e quello del progetto. La variabile `MONDO` seleziona il mondo; i cinque argomenti indicano rispettivamente il numero di droni, l'identificativo iniziale della prova, il numero massimo di nuove prove, l'obiettivo di dieci successi e il numero di droni che devono raggiungere il target per considerare riuscita una prova. In questo esempio, il limite massimo è fissato a 2000 prove. Lo script conta anche i successi già salvati nella serie di risultati del mondo selezionato e interrompe la sequenza al raggiungimento dell'obiettivo o del limite di prove. I riepiloghi delle singole prove sono salvati in file `summary.json`, mentre l'avanzamento della campagna è registrato nel file `progress.log`.

Ogni prova viene gestita da `review_multi.py`, che applica per impostazione predefinita un controllo di stallo di 60 secondi simulati e una soglia di avanzamento verso il target di un metro. I corrispondenti parametri, che possono essere specificati nella chiamata a questo script Python, sono:

```
--stall-abort-s 60 --stall-margin-m 1.0
```

Il drone viene classificato in stallo se, durante tale intervallo, non migliora la distanza dal target rispetto al riferimento di progresso della soglia prevista. Il criterio comprende quindi anche oscillazioni o movimenti senza progresso utile, oltre al caso del veicolo fermo. Nello scenario con cinque droni, il controllo è individuale: lo stallo di un veicolo impedisce il successo completo della prova, ma la simulazione prosegue per osservare l'esito degli altri. La prova termina quando tutti i droni hanno un esito oppure al raggiungimento dei limiti temporali configurati; la campagna può quindi

avviare il tentativo successivo. Il limite di stallo della singola prova e l'obiettivo dei dieci successi della campagna sono pertanto due condizioni distinte.

Gli script di avvio impostano anche alcune variabili d'ambiente necessarie a Gazebo per trovare plugin, modelli e risorse. Tra queste rientrano i percorsi dei plugin Gazebo, delle librerie compilate, dei pacchetti Python e delle risorse SDF.

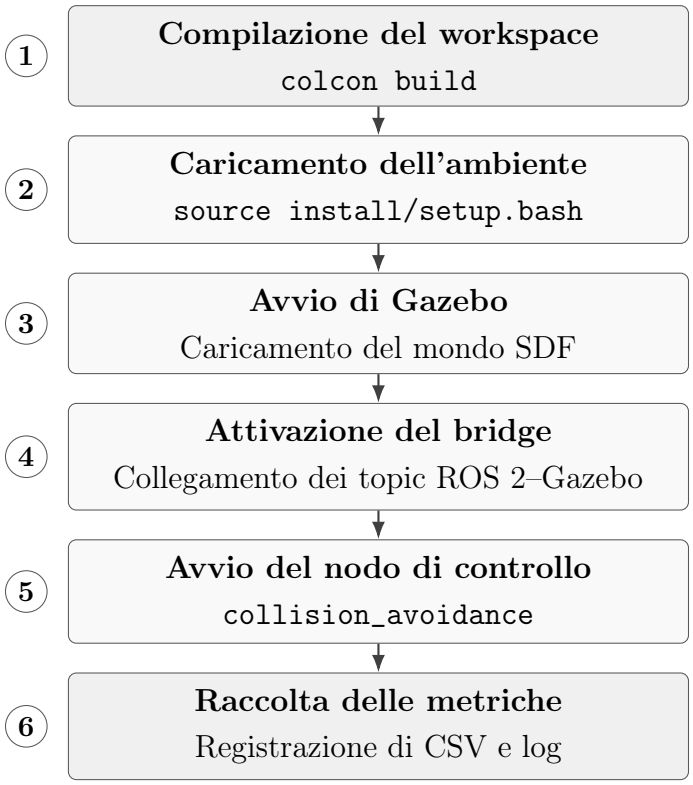


Figura 4.7. *Sequenza tipica di avvio ed esecuzione di una sessione sperimentale.*

Negli scenari multi-drone, la procedura richiede l'avvio di più istanze del nodo, una per ciascun veicolo. Ogni istanza utilizza parametri diversi, come il nome del drone, la posizione iniziale e il target da raggiungere. In questo modo lo stesso codice può essere riutilizzato per configurazioni con un numero diverso di droni.

4.10 Raccolta dei dati sperimentali

La raccolta dei dati sperimentali è stata una parte fondamentale del progetto, poiché ha permesso di valutare in modo quantitativo il comportamento del sistema. Durante le simulazioni sono state registrate diverse metriche, tra cui arrivo al target, tempo di simulazione, distanza percorsa, collisioni, distanza minima dagli ostacoli e separazione minima tra droni negli scenari multi-drone.

Il sistema salva le metriche in file CSV, in modo da poter analizzare successivamente i risultati. Il percorso del file può essere impostato tramite il parametro `csv_path`; se il parametro non viene specificato, viene utilizzato il file `~/ros2_tethys/risultati.csv`. Ogni riga del CSV corrisponde a una prova e contiene, tra gli altri valori, il mondo utilizzato, il numero di ostacoli, le collisioni, la distanza percorsa, l'efficienza, la velocità media, il tempo simulato, il real time factor e l'esito del raggiungimento del target. L'utilizzo del formato CSV facilita l'elaborazione dei dati con script esterni e permette di confrontare più prove tra loro.

Durante l'esecuzione il nodo pubblica inoltre le metriche correnti sul topic ROS 2 `/ca_metrics`. Il messaggio è una stringa in formato JSON e contiene informazioni come il nome del mondo, il numero di ostacoli, le collisioni, la distanza percorsa, la distanza euclidea dal punto iniziale, l'efficienza, la velocità media, il tempo simulato e il real time factor. In questo modo i dati possono essere osservati anche durante la simulazione, mentre il CSV conserva il risultato completo della prova per l'analisi successiva.

Al termine di ogni prova, gli script di revisione leggono le righe del CSV e le raggruppano in base allo scenario e al livello di difficoltà. Da questi dati vengono calcolati il numero di successi e fallimenti, le percentuali di successo, le collisioni e le statistiche aggregate riportate nel Capitolo 5.

Le metriche principali raccolte durante gli esperimenti sono riportate nella Tabella 4.5.

Metrica	**Descrizione**
Arrivo al target	Indica se il drone ha raggiunto il goal entro il raggio stabilito.
Tempo di simulazione	Tempo simulato necessario per raggiungere il target o terminare la prova.
Distanza percorsa	Lunghezza complessiva della traiettoria seguita dal drone.
Collisioni	Numero di contatti fisici registrati durante la prova.
Distanza minima	Distanza minima osservata rispetto agli ostacoli o ai limiti dell'ambiente.
Separazione tra droni	Distanza minima tra veicoli negli scenari multi-drone.
Real time factor	Rapporto tra tempo simulato e tempo reale, utile per valutare il carico della simulazione.

Tabella 4.5. *Metriche principali raccolte durante le campagne sperimentali.*

Oltre alla scrittura dei risultati, sono stati utilizzati script di revisione per ana-

lizzare le prove completate. Tali script permettono di distinguere le prove concluse con successo da quelle fallite, individuare eventuali collisioni e calcolare statistiche aggregate, come mediane, minimi e massimi.

La raccolta dei dati è stata progettata per supportare la valutazione descritta nel capitolo successivo. In particolare, i risultati sperimentali permettono di verificare se il drone riesce a raggiungere il target, se evita collisioni durante gli arrivi e come si comporta in scenari con più veicoli.

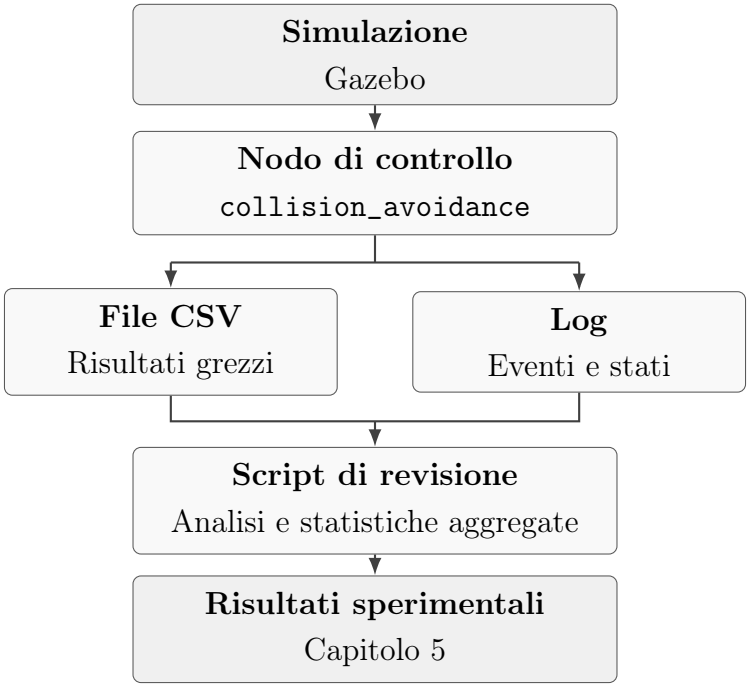


Figura 4.8. *Flusso di raccolta e analisi dei dati sperimentali.*

L'implementazione realizzata integra codice ROS 2, simulazione Gazebo, sensori virtuali, attuatori, planner locale e raccolta dati. Questa integrazione ha permesso di trasformare l'algoritmo di *collision avoidance* in un sistema sperimentabile all'interno di scenari simulati complessi, preparando la base per la valutazione quantitativa presentata nel capitolo successivo.

Capitolo 5

Risultati sperimentali

In questo capitolo vengono presentati i risultati ottenuti durante la campagna sperimentale finale. L'obiettivo degli esperimenti è valutare il comportamento del sistema di *collision avoidance* tridimensionale sviluppato, analizzando la capacità del drone Tethys di raggiungere un target tridimensionale evitando ostacoli presenti nell'ambiente simulato.

La campagna sperimentale è stata organizzata su due scenari principali: uno scenario con un singolo drone e uno scenario con cinque droni presenti contemporaneamente nello stesso ambiente. Per ciascuno scenario sono stati considerati tre livelli di difficoltà, ottenuti aumentando il numero di ostacoli presenti nel mondo simulato. Tutte le prove sono state eseguite utilizzando la versione del nodo disponibile al termine delle campagne; la successiva riorganizzazione dei file ha mantenuto invariato il codice delle funzioni e non modifica i risultati riportati in questo capitolo.

Una prova viene considerata riuscita quando tutti i droni coinvolti nello scenario raggiungono il proprio obiettivo prima dello stallo. Lo stallo viene rilevato quando un drone non si avvicina al goal di almeno un metro per un intervallo di 60 secondi simulati. Nei test con cinque droni, la prova è considerata un successo soltanto se tutti e cinque i veicoli raggiungono il proprio goal prima dello stallo.

5.1 Obiettivi degli esperimenti

Gli esperimenti hanno lo scopo di verificare il comportamento del sistema proposto in condizioni differenti di complessità ambientale. In particolare, la campagna sperimentale mira a valutare la capacità del sistema di raggiungere il target, evitare collisioni, gestire situazioni di blocco e mantenere un comportamento stabile al crescere del numero di ostacoli e del numero di veicoli presenti nell'ambiente.

Il primo obiettivo riguarda la capacità di completare la missione. Poiché il planner sviluppato è locale e non costruisce una mappa globale completa dell'ambiente, è importante verificare se la combinazione tra istogramma sferico, lookahead a corridoi e waypoint locali sia sufficiente per guidare il drone fino al target.

Il secondo obiettivo riguarda la capacità di evitare collisioni e situazioni di blocco. Durante una missione, il drone può trovarsi davanti a pilastri, pareti o altri veicoli. Il sistema deve quindi riconoscere le direzioni non sicure, attivare manovre evasive e, nei casi più critici, frenare o arretrare.

Il terzo obiettivo riguarda la robustezza del sistema al crescere della difficoltà. Per questo motivo sono stati considerati tre livelli per ciascuno scenario. L'aumento del numero di ostacoli rende più probabile la presenza di passaggi stretti, trappole locali e configurazioni nelle quali il drone può rimanere bloccato.

Il quarto obiettivo riguarda il comportamento in presenza di più droni. Nello scenario multi-drone non è stata implementata una logica esplicita di coordinamento tra veicoli: ogni drone esegue il proprio nodo di *collision avoidance* e considera gli altri droni come ostacoli presenti nell'ambiente. Questo permette di valutare il comportamento del sistema in uno scenario più complesso, evidenziando anche i limiti di un approccio puramente locale e non coordinato.

5.2 Setting sperimentale

La campagna finale è stata condotta in simulazione utilizzando ROS 2 e Ignition Gazebo. Gli esperimenti sono stati eseguiti su due tipologie di scenario:

- scenario con un solo drone, in cui un singolo Tethys deve raggiungere il proprio target;
- scenario con cinque droni, in cui più veicoli sono presenti contemporaneamente nello stesso ambiente simulato.

In entrambi i casi, il drone parte da una posizione iniziale e deve raggiungere un target tridimensionale. Gli ostacoli sono rappresentati da pilastri verticali disposti all'interno della vasca. La fisica del veicolo, il sistema di controllo, i sensori simulati e la logica di *collision avoidance* sono rimasti invariati tra i diversi livelli.

Le differenze tra i livelli riguardano principalmente la complessità dello scenario, cioè il numero e la disposizione degli ostacoli. Durante la preparazione dei mondi è stato necessario mantenere gli scenari sufficientemente difficili, ma allo stesso tempo percorribili. Per questo motivo, alcuni mondi sono stati regolati in modo da evitare

configurazioni totalmente impraticabili. Di conseguenza, i livelli devono essere interpretati come scenari sperimentali di difficoltà differente, e non come un confronto puramente teorico basato esclusivamente sul numero di ostacoli.

Per ogni livello, la campagna è stata arrestata al raggiungimento del decimo successo. Le principali metriche raccolte sono:

- numero di tentativi necessari per raggiungere dieci successi;
- numero di successi e fallimenti;
- percentuale di successo;
- collisioni fisiche o blocchi davanti a un ostacolo;
- tempo simulato medio;
- tempo reale medio;
- distanza percorsa dal drone o dai droni.

La voce "collisioni o blocchi davanti a un ostacolo" comprende sia i contatti fisici sia i casi in cui un drone fallisce la prova rimanendo fermo vicino a un pilastro o a una parete nell'ultimo minuto prima dello stallo. Questa distinzione è importante perché molti fallimenti non terminano con una collisione evidente, ma con uno stallo in prossimità di un ostacolo.

5.3 Casi di studio

La campagna sperimentale finale comprende sei casi di studio: tre livelli con un solo drone e tre livelli con cinque droni. La Tabella 5.1 riassume i risultati principali.

Scenario	Livello	Ostacoli	Tentativi	Successi	Successo	Coll./blocchi	Tempo sim. medio
1 drone	Livello 1	30	62	10	16,1%	40	199,0 s
1 drone	Livello 2	35	170	10	5,9%	159	173,3 s
1 drone	Livello 3	40	215	10	4,7%	204	152,0 s
5 droni	Livello 1	40	213	10	4,7%	164	207,3 s
5 droni	Livello 2	50	168	10	6,0%	148	214,6 s
5 droni	Livello 3	60	1018	10	1,0%	1001	210,7 s

Tabella 5.1. *Riepilogo generale della campagna sperimentale finale. La colonna "Coll./blocchi" indica i tentativi con collisione fisica oppure con blocco vicino a un ostacolo o a una parete.*

La Figura 5.1 mostra la percentuale di successo nei sei casi di studio. Il risultato migliore si osserva nello scenario con un solo drone al Livello 1, dove il

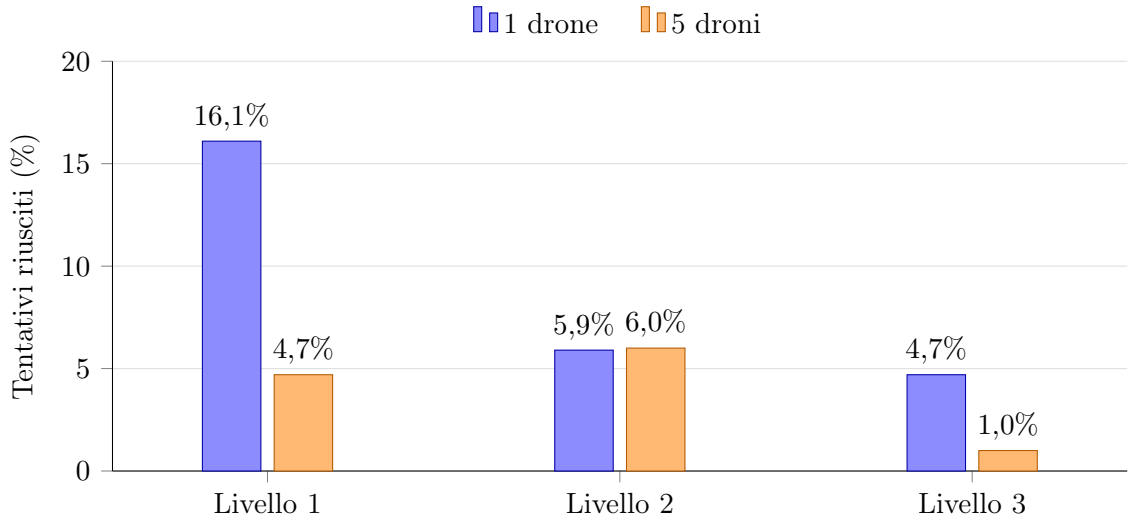


Figura 5.1. *Percentuale di successo nei sei casi di studio della campagna sperimentale.*

sistema raggiunge dieci successi in 62 tentativi, con una percentuale del 16,1%. Nei livelli successivi, la percentuale di successo diminuisce sensibilmente, evidenziando l'aumento della difficoltà.

Nel caso con cinque droni, le percentuali risultano generalmente più basse. Questo dipende dal fatto che una prova viene considerata riuscita soltanto se tutti i droni raggiungono il proprio obiettivo. Di conseguenza, anche il fallimento di un solo veicolo rende fallita l'intera prova.

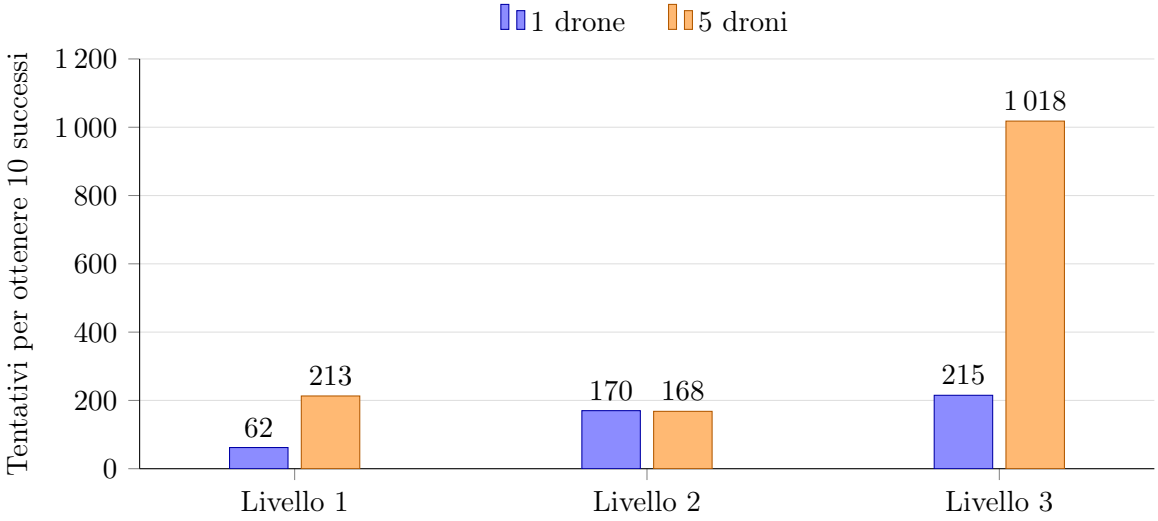


Figura 5.2. *Numero di tentativi necessari per raggiungere dieci successi in ciascun livello.*

La Figura 5.2 evidenzia il numero di tentativi necessari per raggiungere dieci successi. Il caso più critico è il Livello 3 con cinque droni, nel quale sono stati necessari 1018 tentativi. Questo valore mostra chiaramente la difficoltà dello scenario multi-drone più complesso.

5.3.1 Scenario con un drone

Nel primo scenario viene simulato un solo drone Tethys. Questo caso permette di valutare il comportamento del sistema senza interferenze dovute ad altri veicoli.

Nel Livello 1, con 30 ostacoli, il sistema raggiunge dieci successi in 62 tentativi, ottenendo una percentuale di successo pari al 16,1%. Si tratta del risultato migliore tra i tre livelli a singolo drone e mostra che, in uno scenario moderatamente complesso, il sistema è in grado di guidare autonomamente il veicolo fino al target.

Nel Livello 2, con 35 ostacoli, il numero di tentativi necessari per raggiungere dieci successi aumenta a 170. La percentuale di successo scende quindi al 5,9%. Questo peggioramento indica che una maggiore complessità ambientale rende più probabile l'ingresso del drone in configurazioni difficili, nelle quali il planner locale non riesce sempre a individuare una manovra risolutiva.

Nel Livello 3, con 40 ostacoli, il sistema raggiunge dieci successi in 215 tentativi, con una percentuale di successo pari al 4,7%. Il risultato conferma la tendenza osservata nel Livello 2: l'aumento della difficoltà riduce la probabilità di completare la missione.

Nel complesso, lo scenario con un drone mostra che il sistema è funzionante e capace di completare la missione, ma evidenzia anche una forte dipendenza dalla geometria del mondo. I fallimenti non sono distribuiti in modo casuale: spesso si concentrano in zone specifiche dell'ambiente, nelle quali il drone tende a ripetere comportamenti simili e a bloccarsi vicino a un ostacolo o a una parete.

5.3.2 Scenario con cinque droni

Nel secondo scenario sono presenti cinque droni contemporaneamente. Ogni drone esegue il proprio nodo di controllo e deve raggiungere il proprio target. Non è presente una strategia di coordinamento globale: gli altri droni vengono percepiti come ostacoli all'interno dell'ambiente.

Nel Livello 1, con 40 ostacoli, il sistema raggiunge dieci successi in 213 tentativi, con una percentuale di successo pari al 4,7%. Rispetto allo scenario con un solo drone, la difficoltà aumenta sensibilmente, poiché la prova viene considerata riuscita soltanto se tutti e cinque i droni arrivano al goal.

Nel Livello 2, con 50 ostacoli, sono necessari 168 tentativi per ottenere dieci successi, con una percentuale pari al 6,0%. Questo valore è leggermente migliore rispetto al Livello 1. Tale risultato non deve essere interpretato come una contraddizione, ma come conseguenza del fatto che la difficoltà effettiva non dipende esclusivamente dal numero di ostacoli. La disposizione spaziale degli ostacoli e la presenza di trappole locali influenzano in modo significativo il comportamento del sistema.

Nel Livello 3, con 60 ostacoli, la complessità aumenta in modo netto. In questo caso sono stati necessari 1018 tentativi per raggiungere dieci successi, con una percentuale di successo pari all'1,0%. Questo è il caso più difficile dell'intera campagna sperimentale. La presenza simultanea di cinque droni, unita alla complessità dell'ambiente, rende molto più probabile il verificarsi di stalli, blocchi o collisioni.

5.3.3 Analisi dei fallimenti negli scenari multi-drone

Negli scenari con cinque droni, una prova viene considerata riuscita soltanto se tutti i veicoli raggiungono il rispettivo target. Questo criterio rende la valutazione più severa rispetto allo scenario con un solo drone: anche se quattro droni su cinque arrivano correttamente al goal, la prova viene comunque classificata come fallita.

Per comprendere meglio il comportamento del sistema negli scenari multi-drone, è utile analizzare non soltanto il numero complessivo di successi, ma anche quanti droni sono riusciti ad arrivare al target nei tentativi falliti. Questa analisi permette di distinguere i casi in cui il sistema fallisce quasi completamente da quelli in cui la maggior parte dei droni completa comunque la missione.

La Tabella 5.2 riporta la distribuzione dei tentativi falliti nei tre livelli multi-drone. I dati sono riferiti alla campagna ufficiale, cioè ai tentativi considerati fino al raggiungimento del decimo successo.

Scenario	**Fallimenti**	**0/5**	**1/5**	**2/5**	**3/5**	**4/5**
5 droni – Livello 1	203	0	8	59	59	77
5 droni – Livello 2	158	0	7	25	77	49
5 droni – Livello 3	1008	21	127	366	354	140

Tabella 5.2. *Distribuzione dei tentativi falliti negli scenari multi-drone in base al numero di droni arrivati al target. La colonna 4/5 indica i casi in cui un solo drone ha impedito il successo complessivo della prova.*

Nel Livello 1 con cinque droni, su 203 tentativi falliti, in 77 casi sono arrivati quattro droni su cinque. Questo significa che in una parte significativa dei fallimenti il sistema è arrivato molto vicino al successo complessivo, ma un solo drone non ha

raggiunto il target entro il criterio di stallo. In altri 59 casi sono arrivati tre droni e in 59 casi sono arrivati due droni.

Nel Livello 2, su 158 fallimenti, in 49 casi sono arrivati quattro droni su cinque e in 77 casi sono arrivati tre droni su cinque. Anche in questo livello, quindi, molti fallimenti non corrispondono a un fallimento completo del sistema, ma al mancato arrivo di uno o due veicoli.

Nel Livello 3 la situazione risulta più critica. Su 1008 fallimenti, in 366 casi sono arrivati due droni su cinque e in 354 casi sono arrivati tre droni su cinque. I casi in cui arrivano quattro droni sono 140, mentre in 21 tentativi nessun drone è riuscito a completare la missione. Questo conferma che il Livello 3 rappresenta lo scenario multi-drone più difficile dell'intera campagna sperimentale.

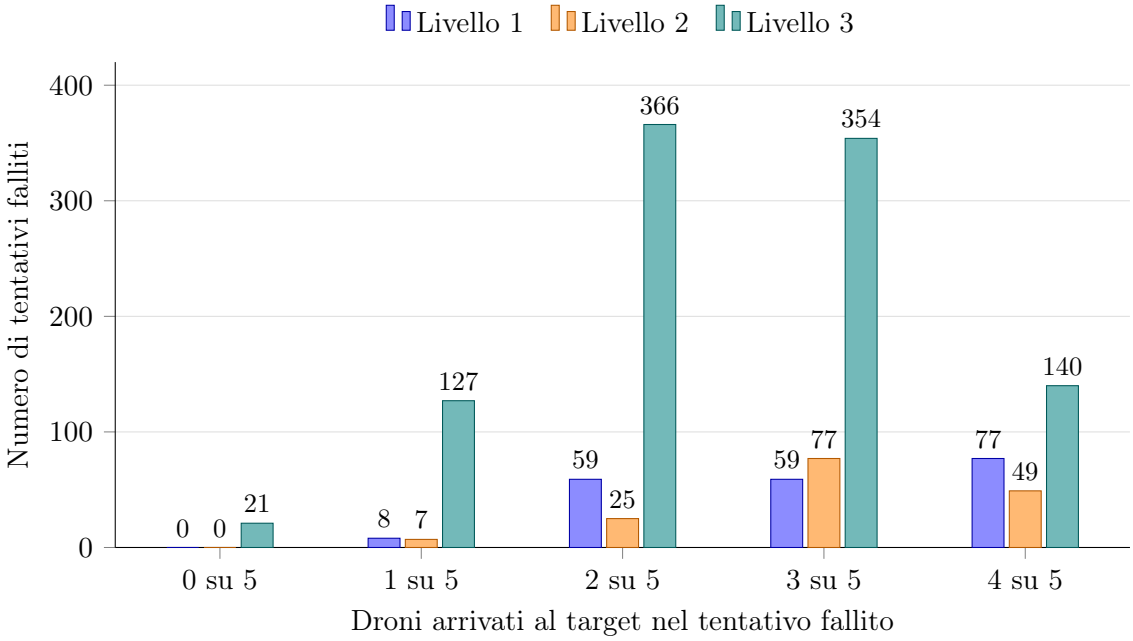


Figura 5.3. *Distribuzione dei tentativi falliti nei livelli multi-drone in base al numero di droni che hanno raggiunto il target.*

La Figura 5.3 evidenzia visivamente la differenza tra i tre livelli multi-drone. Nei Livelli 1 e 2 molti fallimenti avvengono quando tre o quattro droni riescono comunque a raggiungere il target. Nel Livello 3, invece, aumentano notevolmente i casi in cui arrivano soltanto due o tre droni, segno che la difficoltà dello scenario coinvolge più veicoli contemporaneamente.

Questo dato è importante perché mostra che il sistema non fallisce sempre in modo totale. In diversi tentativi, una parte dei droni riesce comunque a completare la missione, mentre uno o più veicoli rimangono bloccati o non raggiungono il goal prima dello stallo. Di conseguenza, il limite principale negli scenari multi-drone non

è soltanto la capacità del singolo drone di evitare gli ostacoli, ma anche la mancanza di una strategia di coordinamento tra veicoli.

5.4 Verifica della correttezza

La verifica della correttezza del sistema non riguarda soltanto il raggiungimento del target, ma anche il rispetto del comportamento atteso durante la missione. In particolare, il sistema deve ricevere correttamente i dati dai sensori, aggiornare la rappresentazione locale dell'ambiente, scegliere una direzione sicura e trasformare tale scelta in comandi per gli attuatori.

I risultati sperimentali confermano che la pipeline implementata funziona. Nei casi di successo, il drone riesce a percepire gli ostacoli, modificare la traiettoria e raggiungere il goal. Questo dimostra che l'integrazione tra sensori simulati, istogramma sferico, planner locale, macchina a stati e controllo degli attuatori produce un comportamento coerente con l'obiettivo del progetto.

La correttezza è stata valutata anche osservando i casi di fallimento. Molti fallimenti non derivano da errori immediati di comunicazione o da assenza di controllo, ma da configurazioni ambientali difficili nelle quali il planner locale non riesce a individuare una manovra risolutiva. In questi casi il drone può rimanere vicino a un ostacolo, oscillare, frenare ripetutamente o non avvicinarsi più al goal fino allo stallo.

Questo comportamento indica che il sistema reagisce agli ostacoli, ma non sempre riesce a superare situazioni locali complesse. Si tratta di un limite tipico dei planner reattivi: in assenza di una pianificazione globale o di una memoria strutturata dell'ambiente, il veicolo può rimanere intrappolato in configurazioni ripetitive.

5.5 Valutazione computazionale

Dal punto di vista computazionale, il sistema è stato progettato per operare in tempo reale simulato. Il nodo di controllo esegue cicli periodici nei quali vengono aggiornati i dati sensoriali, costruito l'istogramma sferico, valutata la presenza di ostacoli e generati i comandi di controllo.

La Tabella 5.3 riporta i tempi medi simulati e reali osservati nei diversi scenari. I tempi sono calcolati sull'intera serie di tentativi, includendo sia prove riuscite sia prove fallite.

Dai risultati emerge che, negli scenari con un solo drone, il tempo reale medio è molto vicino al tempo simulato medio. Questo indica che la simulazione riesce a

mantenere una velocità di esecuzione vicina al tempo reale nei casi a complessità più contenuta.

Negli scenari con cinque droni, invece, il tempo reale medio risulta più alto rispetto al tempo simulato. Questo comportamento è atteso, poiché la simulazione deve gestire più veicoli, più sensori, più nodi di controllo e un numero maggiore di interazioni fisiche. In particolare, l'utilizzo contemporaneo di più sonar simulati aumenta il carico computazionale complessivo.

Scenario	**Livello**	**Tempo simulato medio**	**Tempo reale medio**
1 drone	Livello 1	199,0 s	212,2 s
1 drone	Livello 2	173,3 s	174,2 s
1 drone	Livello 3	152,0 s	152,7 s
5 droni	Livello 1	207,3 s	246,5 s
5 droni	Livello 2	214,6 s	268,2 s
5 droni	Livello 3	210,7 s	277,8 s

Tabella 5.3. *Confronto tra tempo simulato medio e tempo reale medio nei diversi scenari.*

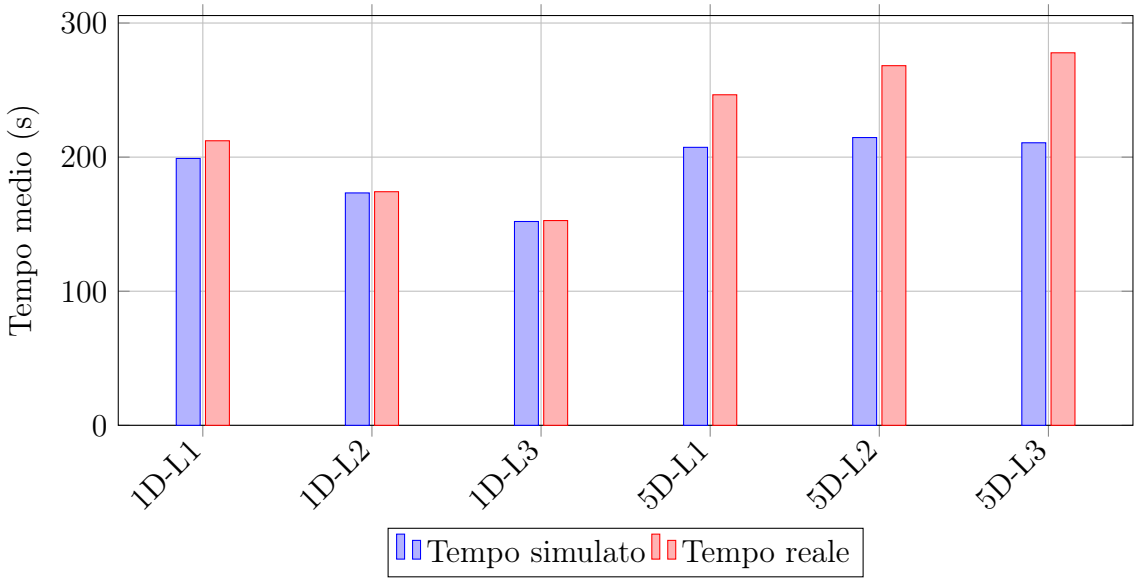


Figura 5.4. *Confronto tra tempo simulato medio e tempo reale medio per ciascun livello.*

5.6 Valutazione tecnica dell'approccio

I risultati mostrano che l'approccio sviluppato è in grado di produrre un comportamento autonomo completo: il drone percepisce ostacoli, costruisce una rappresentazione locale tridimensionale, seleziona una direzione di movimento e genera comandi verso gli attuatori. Questo conferma la validità generale dell'integrazione tra ROS 2, Gazebo, sonar simulati e planner VFH 3D.

Il principale punto di forza del sistema è la capacità di operare senza una mappa globale. Il drone utilizza soltanto informazioni locali provenienti dai sensori e aggiorna continuamente la propria decisione. Questo rende il sistema adatto a scenari nei quali l'ambiente non è noto in anticipo o nei quali una ricostruzione completa dello spazio risulterebbe troppo costosa.

Un secondo punto di forza è l'estensione tridimensionale della rappresentazione. L'uso di un istogramma sferico permette di considerare sia la direzione orizzontale sia la componente verticale degli ostacoli. Anche se nei mondi finali la fuga verticale è stata limitata per mantenere maggiore stabilità, la struttura dell'algoritmo rimane tridimensionale e consente di integrare informazioni provenienti da sonar orientati in più direzioni.

Un terzo elemento positivo è la presenza di meccanismi di sicurezza. La macchina a stati consente di distinguere tra navigazione ordinaria, manovra evasiva e frenata. Inoltre, il sistema include controlli aggiuntivi per gestire quota, profondità, stabilità, prossimità alle pareti e situazioni di emergenza.

I limiti principali emergono invece nei casi di fallimento. Le percentuali di successo, soprattutto nei livelli più difficili, restano basse. Questo indica che il sistema funziona, ma non è ancora sufficientemente robusto per garantire il completamento affidabile della missione in ambienti molto complessi. I fallimenti tendono a concentrarsi in trappole locali ripetibili, dove il drone continua a reagire agli ostacoli ma non riesce a trovare una strategia efficace per uscirne.

La Figura 5.5 mostra che, nei livelli più difficili, la maggior parte dei tentativi falliti è associata a collisioni o blocchi vicino agli ostacoli. Questo conferma che il problema principale non è soltanto evitare l'urto immediato, ma anche evitare configurazioni nelle quali il drone rimane intrappolato senza riuscire a progredire verso il target.

Nel caso multi-drone, il limite più evidente è l'assenza di coordinamento. Poiché ogni veicolo decide autonomamente, senza comunicare con gli altri, possono verificarsi situazioni in cui più droni tentano di utilizzare lo stesso spazio o reagiscono in modo simile allo stesso ostacolo. Questo può generare collisioni o stalli che non

dipendono soltanto dalla presenza degli ostacoli statici, ma anche dall'interazione tra veicoli.

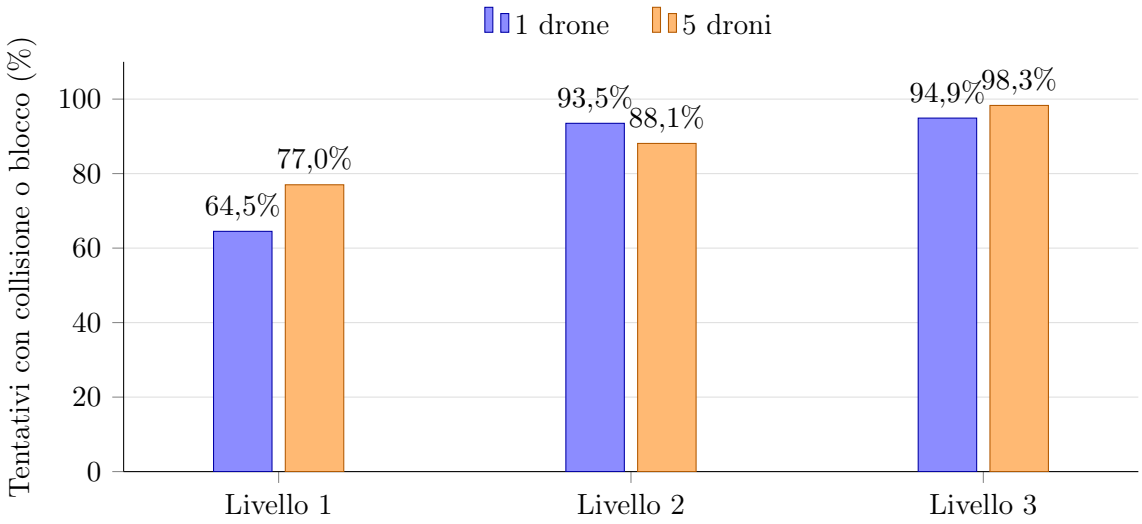


Figura 5.5. *Percentuale di tentativi terminati con collisione fisica oppure con blocco vicino a un ostacolo o a una parete.*

5.7 Discussione dei risultati

La campagna sperimentale evidenzia un comportamento chiaro: il sistema è capace di completare le missioni, ma la sua efficacia diminuisce sensibilmente al crescere della complessità dell'ambiente. Nel caso con un solo drone, la percentuale di successo passa dal 16,1% del Livello 1 al 4,7% del Livello 3. Questo andamento mostra che l'aumento degli ostacoli produce più situazioni di stallo e riduce la probabilità di trovare un percorso libero con un approccio puramente locale.

Nel caso con cinque droni, le percentuali sono generalmente più basse. Il Livello 3 risulta particolarmente critico, con una percentuale di successo dell'1,0%. Questo valore non significa che il sistema non funzioni, ma che lo scenario richiede una robustezza superiore rispetto a quella offerta dal planner locale implementato. La presenza di cinque veicoli rende la prova molto più severa, perché basta il fallimento di un solo drone per invalidare l'intero tentativo.

Un elemento importante da considerare è che la difficoltà effettiva non cresce sempre in modo perfettamente lineare con il numero di ostacoli. Il Livello 2 con cinque droni, ad esempio, presenta una percentuale di successo leggermente superiore rispetto al Livello 1. Questo risultato conferma che la disposizione degli ostacoli e la presenza di trappole locali incidono più del solo numero totale di ostacoli.

L'analisi dei fallimenti negli scenari multi-drone permette inoltre di osservare che molti tentativi non falliscono in modo totale. Nei Livelli 1 e 2, diversi fallimenti avvengono quando tre o quattro droni raggiungono comunque il target. In questi casi il problema è spesso concentrato su uno o due veicoli che rimangono bloccati, entrano in stallo o non riescono a completare il percorso entro il tempo previsto.

Nel Livello 3 con cinque droni, invece, i fallimenti sono più distribuiti sull'intero gruppo. Il numero elevato di casi con due o tre droni arrivati mostra che la difficoltà dello scenario non riguarda soltanto un singolo veicolo, ma coinvolge più traiettorie contemporaneamente. Questo conferma che, in assenza di coordinamento, la complessità multi-drone può diventare rapidamente dominante.

I dati sulle collisioni e sui blocchi confermano questa interpretazione. Nei livelli più difficili, gran parte dei fallimenti è associata a situazioni in cui il drone rimane vicino a ostacoli o pareti, senza riuscire ad avanzare verso il target. Questo suggerisce che il problema principale non è soltanto evitare l'urto immediato, ma anche evitare stati di navigazione nei quali il drone rimane intrappolato.

Dal punto di vista progettuale, il risultato è comunque significativo. Il sistema sviluppato rappresenta una pipeline completa di *collision avoidance* tridimensionale per un AUV simulato. Esso integra percezione, pianificazione locale, controllo, gestione degli stati e raccolta delle metriche. La campagna sperimentale non dimostra soltanto i casi di successo, ma mette anche in evidenza i limiti tecnici dell'approccio, fornendo indicazioni precise per gli sviluppi futuri.

In conclusione, i risultati mostrano che il metodo proposto è adatto come base sperimentale per la navigazione autonoma locale di un drone subacqueo, ma richiede ulteriori miglioramenti per aumentare la percentuale di successo negli ambienti più complessi. In particolare, risultano necessari meccanismi più robusti per uscire dalle trappole locali, una gestione più avanzata del multi-drone e una strategia di pianificazione capace di combinare reattività locale e visione più ampia dell'ambiente.

Capitolo 6

Conclusioni

In questa tesi è stato presentato lo sviluppo di un sistema di *collision avoidance* tridimensionale per il veicolo subacqueo Tethys LRAUV, simulato in ambiente ROS 2 e Ignition Gazebo. Il lavoro è nato dall'esigenza di progettare un algoritmo capace di guidare autonomamente un drone subacqueo verso un obiettivo tridimensionale, evitando ostacoli presenti nell'ambiente.

Il sistema sviluppato si basa su un planner locale ispirato all'algoritmo *Vector Field Histogram*, esteso al caso tridimensionale mediante una rappresentazione sferica dello spazio circostante. I dati provenienti dai sonar simulati vengono trasformati in un istogramma basato su azimuth ed elevazione, che permette al drone di valutare la presenza di ostacoli nelle diverse direzioni e di scegliere una traiettoria localmente sicura.

L'algoritmo è stato implementato in un pacchetto ROS 2 scritto in C++, integrato con il modello del drone Tethys all'interno del simulatore Gazebo. Il sistema comprende la ricezione delle nuvole di punti, la fusione dei dati sensoriali, la costruzione dell'istogramma sferico, la ricerca di corridoi percorribili, la generazione di waypoint locali e l'invio dei comandi alle pinne e al propulsore.

La campagna sperimentale finale ha permesso di valutare il sistema in due scenari principali: uno con un solo drone e uno con cinque droni. Per ciascuno scenario sono stati considerati tre livelli di difficoltà. I risultati mostrano che il sistema è in grado di completare missioni autonome, ma evidenziano anche una riduzione significativa dell'affidabilità negli scenari più complessi.

6.1 Sintesi del lavoro svolto

Il progetto può essere riassunto in tre fasi principali: studio del problema, progettazione del metodo e valutazione sperimentale.

La prima fase ha riguardato lo studio del contesto applicativo e degli strumenti utilizzati. Sono stati analizzati i veicoli autonomi subacquei, il problema della navigazione autonoma, la *collision avoidance*, l'ambiente ROS 2, la simulazione robotica con Gazebo, i sensori sonar e l'algoritmo VFH. Questa fase ha permesso di definire le basi teoriche e tecnologiche necessarie per lo sviluppo del sistema.

La seconda fase ha riguardato la progettazione dell'approccio. Il problema dell'evitamento ostacoli è stato affrontato tramite una rappresentazione locale tridimensionale dell'ambiente. Ogni punto rilevato dai sonar viene associato a una direzione nello spazio e contribuisce alla costruzione di un istogramma sferico. A partire da questa rappresentazione, il planner valuta le direzioni occupate e quelle libere, cercando una traiettoria che consenta al drone di proseguire verso il target.

La terza fase ha riguardato l'implementazione e il testing. È stato realizzato il pacchetto `collision_avoidance`, contenente il nodo principale di controllo. Il nodo riceve i dati dai sonar simulati, aggiorna lo stato del veicolo, costruisce l'istogramma, seleziona una direzione sicura e pubblica i comandi per gli attuatori. Sono inoltre stati preparati mondi simulati, script di avvio, bridge tra ROS 2 e Gazebo e strumenti per la raccolta dei risultati.

Dal punto di vista sperimentale, il sistema è stato testato su due scenari: uno con un solo drone e uno con cinque droni. In entrambi i casi sono stati considerati tre livelli di difficoltà. I risultati hanno mostrato che il sistema riesce a completare missioni autonome in tutti i livelli considerati, ma con percentuali di successo molto diverse.

Nel caso con un solo drone, il livello meno complesso ha ottenuto il risultato migliore, con una percentuale di successo pari al 16,1%. Nei livelli successivi, la percentuale è scesa al 5,9% e al 4,7%. Nel caso con cinque droni, la difficoltà è risultata maggiore: il Livello 1 ha ottenuto il 4,7%, il Livello 2 il 6,0% e il Livello 3 l'1,0%. Questi risultati indicano che il sistema è funzionante, ma che la robustezza diminuisce sensibilmente negli scenari più complessi.

Un aspetto importante emerso dalla campagna sperimentale è che molti fallimenti non corrispondono a una collisione immediata, ma a situazioni di stallo vicino a ostacoli o pareti. Il drone riesce a reagire localmente alla presenza di ostacoli, ma in alcuni casi rimane bloccato in configurazioni ripetitive senza riuscire a trovare una manovra efficace per proseguire verso il target.

Nel caso multi-drone, il problema è ancora più evidente. Una prova è considerata riuscita soltanto se tutti i droni raggiungono il goal. Di conseguenza, anche un singolo drone bloccato rende fallito l'intero tentativo. L'analisi dei fallimenti ha mostrato che, in diversi casi, tre o quattro droni riescono comunque ad arrivare al target, mentre uno o due veicoli rimangono in stallo. Questo conferma che la gestione multi-drone richiede strategie più avanzate rispetto alla semplice percezione degli altri veicoli come ostacoli.

6.2 Limiti del sistema sviluppato

Il primo limite del sistema riguarda la natura locale del planner. L'algoritmo prende decisioni sulla base delle informazioni sensoriali disponibili nell'intorno del drone, senza costruire una mappa globale dell'ambiente. Questo approccio permette di reagire rapidamente agli ostacoli, ma non garantisce sempre la scelta migliore nel lungo periodo.

In ambienti complessi, il drone può entrare in trappole locali. In questi casi, il sistema continua a rilevare ostacoli e a generare comandi, ma non riesce a trovare una traiettoria risolutiva. Il drone può quindi rallentare, frenare, oscillare o rimanere vicino a un pilastro senza riuscire ad avanzare verso il target. Questo comportamento è stato osservato soprattutto nei livelli più difficili.

Un secondo limite riguarda la mancanza di pianificazione globale. Il sistema non dispone di una rappresentazione completa del mondo e non può quindi pianificare un percorso alternativo più ampio. Una scelta che appare sicura localmente può portare il veicolo verso una zona senza uscita o verso una configurazione difficile da superare.

Un terzo limite riguarda lo scenario multi-drone. Nel sistema sviluppato non esiste una comunicazione diretta tra i veicoli, né una logica di priorità o coordinamento. Ogni drone esegue il proprio nodo di controllo e considera gli altri droni come ostacoli mobili. Questa scelta semplifica l'architettura, ma non impedisce situazioni in cui due o più droni scelgono manovre incompatibili o cercano di attraversare lo stesso passaggio.

Un ulteriore limite riguarda il controllo del veicolo. Il Tethys è un sistema subacqueo con dinamica più complessa rispetto a un robot terrestre. Le pinne, il propulsore, il pitch, la quota e la velocità influenzano il comportamento durante le manovre evasive. Anche quando la direzione scelta dal planner è corretta, il drone può impiegare tempo per orientarsi o per stabilizzarsi, aumentando il rischio di stallo o collisione.

Va inoltre considerato che la validazione è stata svolta interamente in simulazione. Gazebo permette di riprodurre scenari complessi e ripetibili, ma non rappresenta tutte le incertezze di un ambiente reale. In un contesto operativo reale sarebbero presenti rumore sensoriale più complesso, correnti marine, ritardi, incertezze di localizzazione e possibili errori nei modelli fisici.

Infine, il raggiungimento del goal è stato valutato tramite una sfera tridimensionale attorno al target. Questo criterio è utile per stabilire in modo automatico se una prova è riuscita, ma non garantisce necessariamente una convergenza perfetta alla quota desiderata o alla traiettoria ideale. In alcuni casi il drone può avvicinarsi molto al target, ma non riuscire a entrare nella sfera entro il criterio di stallo.

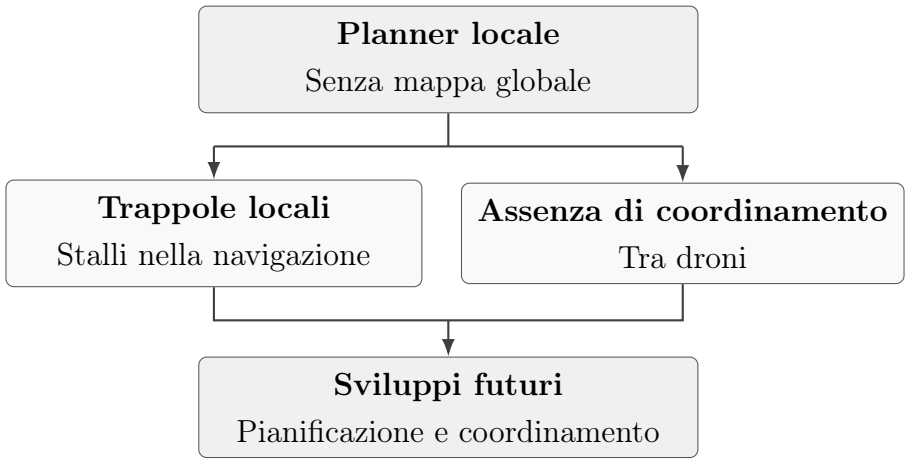


Figura 6.1: Sintesi dei principali limiti emersi dagli esperimenti e delle direzioni di sviluppo future.

6.3 Sviluppi futuri

I risultati ottenuti permettono di individuare diverse direzioni di miglioramento.

Un primo sviluppo riguarda l'integrazione di una pianificazione globale o semi-globale. Il sistema attuale è basato su decisioni locali, aggiornate a ogni ciclo di controllo. L'aggiunta di una rappresentazione più ampia dell'ambiente potrebbe aiutare il drone a evitare trappole locali e a scegliere traiettorie più efficaci nel lungo periodo.

Un secondo miglioramento riguarda la memoria degli ostacoli. Attualmente il sistema utilizza principalmente informazioni locali e meccanismi di memoria limitata. Una memoria più strutturata potrebbe permettere al drone di riconoscere zone già visitate o configurazioni problematiche, evitando di ripetere manovre inefficaci.

Un terzo sviluppo riguarda la gestione multi-drone. Per migliorare gli scenari con più veicoli sarebbe utile introdurre strategie di coordinamento. Ad esempio, i droni potrebbero condividere informazioni sulla propria posizione, assegnarsi priorità, mantenere distanze minime più robuste o seguire regole di precedenza. Anche una strategia semplice basata su corsie o separazione spaziale potrebbe ridurre il numero di interferenze reciproche.

Un quarto miglioramento riguarda la scelta dei waypoint locali. Nei casi di stallo, il drone spesso continua a scegliere manovre che sono localmente sicure, ma non realmente utili per uscire dalla configurazione problematica. Un criterio di selezione più avanzato, capace di valutare anche il progresso verso il target e non soltanto la distanza dagli ostacoli, potrebbe aumentare la percentuale di successo.

Un ulteriore sviluppo riguarda il controllo del veicolo. Il comportamento del Tethys dipende dalla combinazione tra spinta, pinne, assetto e profondità. Un controllo più raffinato della quota, del pitch e della frenata potrebbe migliorare la stabilità durante manovre evasive, retromarce e situazioni di emergenza.

Sarebbe inoltre utile ampliare la campagna sperimentale. I test svolti hanno permesso di raccogliere una quantità significativa di dati, ma ulteriori scenari potrebbero rendere la valutazione più completa. In particolare, si potrebbero considerare ambienti con ostacoli dinamici, corridoi più realistici, pareti irregolari o target multipli.

Infine, un possibile sviluppo riguarda il passaggio verso simulazioni più realistiche o test su piattaforme fisiche. Prima di un'applicazione reale, sarebbe necessario introdurre modelli sensoriali più accurati, rumore più realistico, incertezze di localizzazione e condizioni ambientali variabili. Questo permetterebbe di valutare la robustezza del sistema in condizioni più vicine a quelle operative.

6.4 Considerazioni finali

I risultati ottenuti mostrano la fattibilità di un sistema di *collision avoidance* tridimensionale per droni subacquei simulati. L'approccio adottato integra percezione, pianificazione locale e controllo in un'unica pipeline, permettendo al drone Tethys di navigare autonomamente in ambienti con ostacoli.

I risultati sperimentali mostrano che il sistema è in grado di completare missioni in tutti gli scenari considerati. Allo stesso tempo, la campagna di test evidenzia limiti importanti, soprattutto nei livelli più complessi e negli scenari multi-drone. Le basse percentuali di successo non annullano il valore del lavoro svolto, ma indicano chiaramente quali aspetti devono essere migliorati.

Il contributo principale della tesi consiste quindi nella progettazione e simulazione di una pipeline completa per la *collision avoidance* tridimensionale, basata su un'estensione del VFH e integrata in ROS 2 e Gazebo. Il sistema rappresenta una base sperimentale utile per ulteriori sviluppi, in particolare verso planner più robusti, coordinamento multi-drone e simulazioni più realistiche.

In conclusione, il progetto ha permesso di realizzare un sistema funzionante, testarlo in scenari progressivamente più complessi e individuare in modo preciso i principali limiti tecnici dell'approccio. Questi risultati costituiscono un punto di partenza concreto per il miglioramento della navigazione autonoma di veicoli subacquei in ambienti tridimensionali.

Bibliografia

[1] J. Borenstein e Y. Koren, "The Vector Field Histogram—Fast Obstacle Avoidance for Mobile Robots", *IEEE Transactions on Robotics and Automation*, vol. 7, n. 3, pp. 278–288, 1991, doi: 10.1109/70.88137.

[2] ROS 2, *ROS 2 Documentation: Humble*, documentazione ufficiale, repository, ramo `humble`. `https://github.com/ros2/ros2_documentation/tree/humble`. Consultato il 9 ottobre 2026.

[3] Gazebo, *Gazebo Fortress Tutorials*, documentazione ufficiale. `https://gazebosim.org/docs/fortress/tutorials/`. Consultato il 9 ottobre 2026.

[4] Gazebo, *Underwater vehicles*, documentazione ufficiale di Ignition Gazebo 6 (Fortress). `https://gazebosim.org/api/gazebo/6/underwater_vehicles.html`. Consultato il 9 ottobre 2026.

[5] Gazebo, *ros_gz_bridge*, documentazione del pacchetto nel repository ufficiale `ros_gz`, ramo `humble`. `https://github.com/gazebosim/ros_gz/tree/humble/ros_gz_bridge`. Consultato il 9 ottobre 2026.

[6] Monterey Bay Aquarium Research Institute (MBARI), *Long-Range AUV (LRAUV)*, sito istituzionale. `https://www.mbari.org/technology/long-range-auv-lrauv/`. Consultato il 9 ottobre 2026.

[7] S. M. LaValle, *Planning Algorithms*, Cambridge University Press, 2006. Versione online disponibile sul sito dell'autore: `https://lavalle.pl/planning/`.

[8] T. I. Fossen, *Handbook of Marine Craft Hydrodynamics and Motion Control*, 2ª ed., Wiley, 2021. Scheda bibliografica sul sito dell'autore: `https://www.fossen.biz/publications/books.php`.

Ringraziamenti

Desidero ringraziare la mia famiglia per il sostegno e l'affetto con cui mi ha accompagnato durante questo percorso. Grazie per aver creduto in me, per avermi incoraggiato nei momenti di difficoltà e per aver condiviso con me la gioia di ogni traguardo.

Una dedica speciale va ai miei nonni, ai quali dedico questa tesi. Il vostro affetto e i vostri insegnamenti sono una parte preziosa della mia vita. Questo traguardo porta con sé anche ciò che mi avete trasmesso e rappresenta un'occasione per dirvi, con gratitudine, quanto siete importanti per me.